Tema 4 · Búsqueda heurística avanzada

Manual de estudio · Módulo II: Agentes basados en búsqueda
Material docente URJC, curso 2026–2027

Cómo construir, aprender y comparar funciones heurísticas; cómo ejecutar A* y cómo reducir sus necesidades de memoria o decidir mientras se actúa. Los ejercicios se desarrollan dentro de la explicación del procedimiento que enseñan.

Convención esencial. En estas fuentes, **h(n)** es el coste real mínimo restante y **h*(n)** es su estimación. Muchos libros usan los símbolos al revés. Todo el manual mantiene la notación del profesor. Las explicaciones de «Aprendizaje-h-estrella» y «h-estrella-mas-informada» se han elaborado para este manual: no son transcripciones de los vídeos, que no se han facilitado.

Índice

1. Fundamentos y condiciones de A* 2

2. Aprendizaje de h*: mejorar al buscar 3

3. Laberintos: heurística y aprendizaje 8

4. Diseño mediante problemas relajados 11

5. Una h* más informada: significado y límites 14

6. Aplicación: comparar A* en el 8-puzzle 17

7. Modelar bien importa tanto como la heurística 21

8. Rutas con batería: ampliar el estado 23

9. Complejidad y alternativas con menos memoria 26

10. Restringir la búsqueda: islas y jerarquías 28

11. Búsqueda en línea y horizonte limitado 29

12. Criterios para resolver y comprobar un problema nuevo 34

Las figuras procedentes de las fuentes se recortan o reconstruyen. Los valores sucesivos de las animaciones se explican mediante tablas y estados estáticos. Las precisiones técnicas aparecen junto al concepto afectado.

1. Fundamentos y condiciones de A*

Un problema de búsqueda especifica estados, estado inicial, prueba de meta, acciones aplicables y coste de cada transición. Un **estado** describe una situación del problema; un **nodo** del árbol representa una llegada a ella por un camino concreto. Dos nodos pueden contener el mismo estado y tener costes g diferentes. Esta distinción determina cómo se almacenan las heurísticas aprendidas y cómo se filtran repeticiones.

Símbolo	Significado
g(n)	Coste acumulado del camino representado por n desde el inicio. Se conoce exactamente para ese camino.
h(n)	Menor coste real desde el estado de n hasta cualquier meta. Puede ser infinito si no hay solución desde él.
h*(n)	Estimación del coste restante; normalmente depende del estado y de la meta, no del camino de llegada.
f*(n) = g(n) + h*(n)	Estimación del coste total de una solución que continúe desde ese nodo.
f(n) = g(n) + h(n)	Coste de la mejor continuación del camino concreto. No implica que g sea la mejor llegada posible al estado.

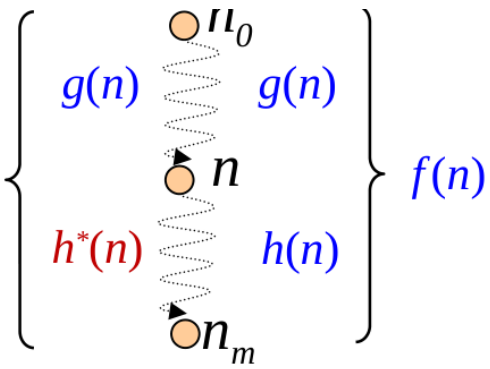


Figura 1. Descomposición del coste del camino: tramo conocido g y tramo restante h, aproximado por h*. Recorte del diagrama docente.

1.1. Optimismo y consistencia

> Optimismo: 0 ≤ h*(n) ≤ h(n). En una meta: h*(n) = 0.

Una heurística optimista, o admisible, nunca promete un coste mínimo mayor que el real. Puede infravalorarlo mucho: h*=0 es optimista con costes no negativos. Optimista no significa precisa, ni significa que toda decisión local vaya en la dirección correcta.

> Consistencia: h*(n) ≤ c(n,n′) + h*(n′), para toda transición n → n′.

La estimación no puede caer en un paso más que el coste de ese paso. Restando h*(n′) se obtiene la forma del solucionario: h*(n)−h*(n′)≤c(n,n′). Al sumar g(n′)=g(n)+c(n,n′),

resulta $f^*(n') \geq f^*(n)$ a lo largo de un camino. Esta monotonía es la base de las podas por horizonte.

Si la heurística vale cero en las metas, la consistencia implica optimismo: se encadena la desigualdad por un camino óptimo $n_0,\ldots,n_m$. Los términos intermedios se cancelan y queda $h^*(n_0) \leq \Sigma c(n_i,n_{i+1}) + h^*(n_m) = h(n_0)$. El recíproco es falso: con una arista X→Y de coste 1 y otra Y→G de coste 4, los valores $h^*(X)=5$, $h^*(Y)=0$ y $h^*(G)=0$ son optimistas, pero $5 \leq 1+0$ es falso.

1.2. Ejecución que debes saber reconstruir

Se introduce el nodo inicial en **Abierta** con g=0. Se extrae un nodo de f* mínimo, se comprueba si es meta y, si no lo es, se generan sus sucesores. Para cada sucesor se calcula $g'=g+c$, después h* y finalmente $f^*=g'+h^*$. Se incorporan los nuevos nodos y se ordena de nuevo Abierta. El padre guardado permite reconstruir la solución al seleccionar una meta.

> **Generar una meta no basta para terminar A*.** Hay que seleccionarla como nodo de f* mínimo. En las figuras docentes la meta suele llevar el último número de «expansión», aunque no se generan sus sucesores. Aquí se distingue selección de meta y expansión efectiva.

El desempate se aplica sólo entre valores mínimos iguales. Debe respetarse el del enunciado: en el 8-puzzle se busca el peor caso, y en el laberinto se favorece la cercanía a una meta. Si persiste el empate, puede haber varias trazas correctas. Al evitar ciclos generales se compara el estado completo. Si aparece una llegada más barata, hay que actualizarla; con heurísticas inconsistentes puede ser necesario reabrir estados cerrados.

1.3. Qué garantizan realmente las propiedades

A* en árbol es óptimo con una heurística optimista y prueba de meta al extraer. En grafo, esta garantía exige gestionar correctamente mejoras de g; la consistencia permite cerrar un estado de forma definitiva en la variante habitual. La completitud en espacios infinitos requiere condiciones adicionales, como ramificación finita y costes de paso acotados inferiormente por $\varepsilon>0$, junto con una heurística no negativa. En un espacio finito con gestión correcta de repetidos, las condiciones pueden ser más débiles. [R1]

La intuición de optimalidad es que, mientras quede una solución mejor de coste C*, habrá en la frontera un prefijo de esa solución con $f^* \leq C^*$. Una meta peor tiene $f^*=g>C^*$ y no puede ser seleccionada antes. Este razonamiento necesita que ese prefijo no se haya eliminado incorrectamente.

2. Aprendizaje de h*: mejorar al buscar

El aprendizaje usa búsquedas anteriores o expansiones de la búsqueda actual para corregir estimaciones del coste restante. Si se mantiene el mismo dominio, sus costes y la misma meta, lo aprendido desde un origen puede reutilizarse desde otro. Se aprende una tabla de valores por **estado**; el camino y g siguen siendo propios de cada nodo.

2.1. La regla y por qué usa el mínimo

$$h^*_{\text{nuevo}}(s) = \min \{ c(s,t) + h^*(t) : t \in \text{expandir}(s) \}.$$

Toda solución desde s empieza por algún sucesor t. El término c(s,t)+h*(t) estima el coste de esa alternativa completa: primer paso real más resto estimado. Se toma el mínimo porque basta encontrar la alternativa más barata. No se suman las alternativas, no se toma su máximo y no se incluye el coste g de haber llegado a s.

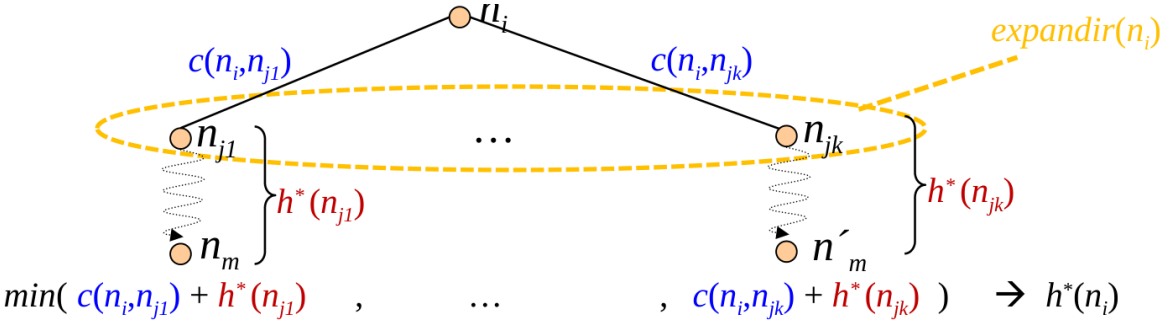


Figura 2. Actualización hacia el padre a partir de todos sus sucesores. Recorte del esquema docente; los costes de arista y las estimaciones de las hojas tienen papeles distintos.

Por ejemplo, si las tres alternativas tienen costes de paso 2, 5 y 1, y estimaciones restantes 4, 0 y 8, sus valores son 6, 5 y 9. Se aprende h*(s)=5. La mejor estimación pasa por el segundo sucesor, aunque su primer paso sea el más caro. El valor 5 sigue siendo una estimación: si ese sucesor realmente requiere 7 unidades más, su alternativa cuesta 12.

La garantía de optimismo se demuestra usando un sucesor t_o de algún camino óptimo. Como $h^*(t_o) \leq h(t_o)$, el mínimo de todas las alternativas es como mucho $c(s,t_o)+h^*(t_o)$, que a su vez es como mucho $c(s,t_o)+h(t_o)=h(s)$. Es importante considerar todos los sucesores legales: tomar el mínimo sólo de una selección arbitraria puede eliminar precisamente la alternativa óptima e invalidar la prueba.

2.2. Secuencia completa de una actualización

El método docente empieza con h*(s)=0 para todos los estados y mantiene h*(meta)=0. Al expandir s, consulta los sucesores, evalúa cada coste más su estimación, actualiza la entrada de s y conserva la tabla para búsquedas posteriores. Si s es una meta no se aplica el mínimo de sus sucesores. Si un estado no meta carece de sucesores y no puede llegar a una meta, se interpreta min□ como +∞.

Partiendo de cero con costes no negativos, estas actualizaciones completas no disminuyen los valores y preservan la consistencia. Para las aristas salientes de s, el nuevo mínimo es menor o igual que cada c+h*. Para las entrantes, aumentar h*(s) sólo facilita sus desigualdades. Una heurística inicialmente sólo optimista no ofrece por sí sola esta monotonía; una ampliación habitual usa max(h* anterior, mínimo nuevo) para no perder información, pero no es la regla literal de las diapositivas.

No se aprende h exacta de todos los estados en una sola búsqueda. La información se propaga cuando se vuelven a actualizar los estados necesarios. Tampoco basta repetir siempre la misma búsqueda para garantizar que se visite todo el dominio. La convergencia global requiere hipótesis de exploración y del problema. El coste práctico de este método es almacenar la tabla de estados aprendidos.

2.3. Actualizar las copias que ya están en Abierta

Si una ciudad aparece en varios nodos, todos comparten h*(ciudad), pero cada nodo conserva su g. Supón que dos copias tienen g=10 y g=16 y que h* cambia de 3 a 8. Sus prioridades deberían pasar de 13 a 18 y de 19 a 24. Hay que reordenar Abierta después del cambio; cambiar sólo la tabla sin reparar las prioridades no produce la variante actualizada.

Dos variantes presentes en las fuentes. Las diapositivas 7–9 actualizan f* de las hojas existentes. El ejercicio 3c conserva sus valores antiguos y explica la actualización como alternativa. Ambas trazas se mantienen separadas. El h* impreso junto a un nodo puede ser el consultado al generarlo, distinto del actual en la tabla.

2.4. Primera búsqueda: de A a L

En el mapa de las diapositivas se usan A–Z=75, A–T=118, A–S=140, Z–O=71, O–S=151, T–L=111, S–R=80, S–F=99, R–P=97 y R–C=146. Las carreteras son bidireccionales. Se permiten copias y se actualiza f* en las hojas. La meta L permanece en cero.

Paso	Estado	g	h* anterior → nueva	Hijos: estado, g+h*=f*
1	A	0	0 → 75	Z: 75+0=75; T: 118+0=118; S: 140+0=140
2	Z	75	0 → 71	A: 150+75=225; O: 146+0=146
3	T	118	0 → 111	A: 236+75=311; L: 229+0=229
4	S	140	0 → 80	A: 280+75=355; O: 291+0=291; R: 220+0=220; F: 239+0=239
5	O	146	0 → 142	Z: 217+71=288; S: 297+80=377
6	R	220	0 → 97	C: 366+0=366; S: 300+80=380; P: 317+0=317
7	A	150	75 → 146	Z: 225+71=296; T: 268+111=379; S: 290+80=370

Al expandir O, se aprende min(71+71,151+80)=142. La copia de O generada desde S cambia de f*=291 a 291+142=433. Más tarde se expande la copia de A alcanzada por A–Z–A, con g=150: aprende min(75+71,118+111,140+80)=146. Sus otras copias abiertas también se corrigen. Finalmente se selecciona L con g=229; el camino devuelto es A–T–L, no la secuencia de estados expandidos.

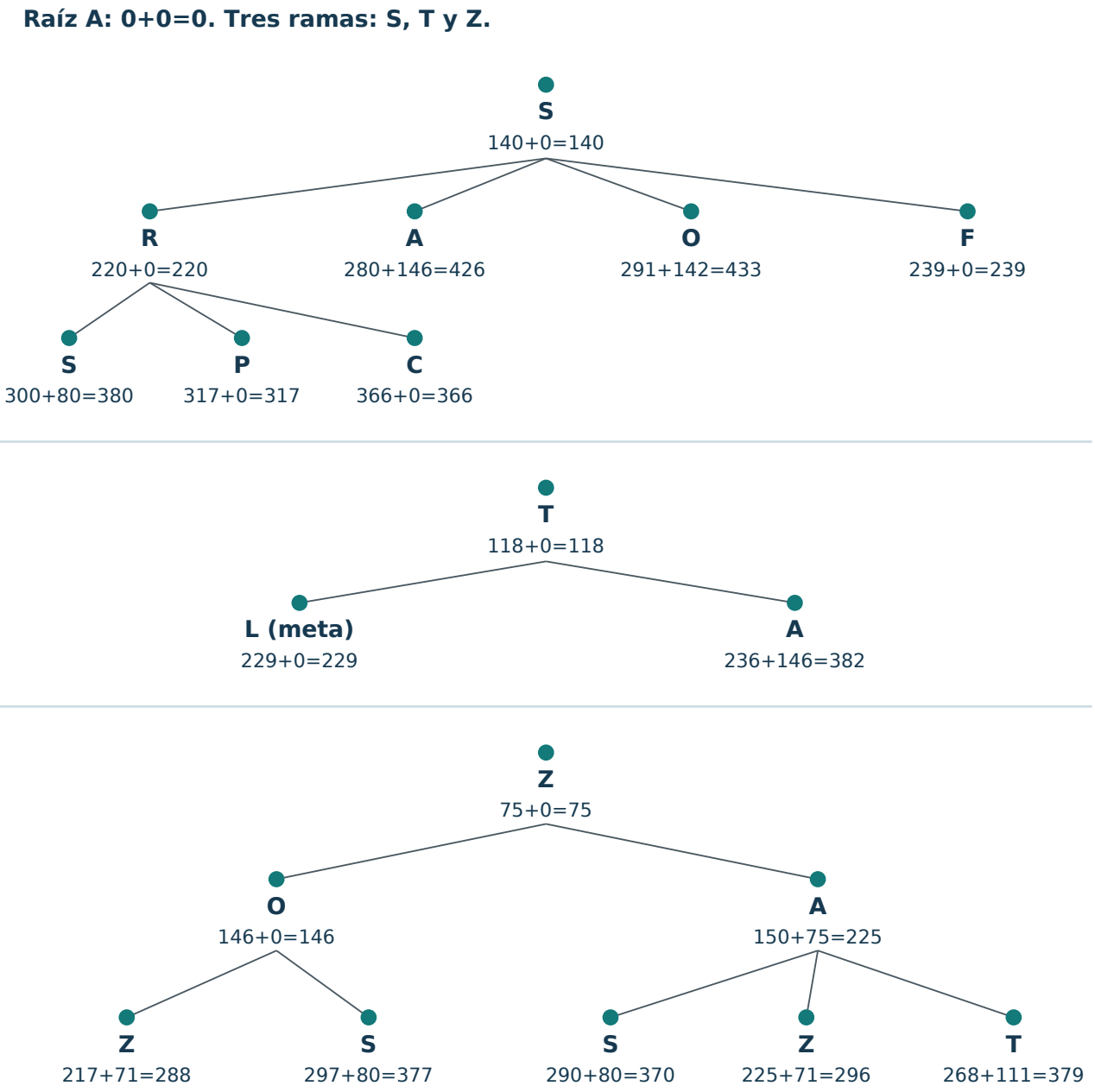


Figura 3. Reconstrucción del árbol A→L, dividido por sus tres ramas iniciales. Cada etiqueta indica g+h*=f*. Las hojas muestran su prioridad final actualizada; los nodos expandidos, la prioridad con la que fueron seleccionados. Se conservan todos los nodos del ejemplo.

2.5. Segunda y tercera búsquedas: reutilización

La segunda búsqueda empieza en Z con la tabla anterior. Primero Z aprende 213=min(75+146,71+142). O aprende 231=min(71+213,151+80). A aprende 220=min(75+213,118+111,140+80). S aprende 99=min(140+220,151+231,80+97,99+0). Esta última actualización cambia la copia abierta de S con g=222 de f*=302 a f*=321, y evita expandirla antes que T, cuyo f* es 304.

Búsqueda	Orden de expansión antes de meta	Ruta y coste	Valores aprendidos al acabar
A→L, primera	A, Z, T, S, O, R, A	A–T–L; 229	A=146, Z=71, T=111, S=80, O=142, R=97
Z→L, segunda	Z, O, A, S, T	Z–A–T–L; 304	A=220, Z=213, T=111, S=99, O=231, R=97
A→L, tercera	A, T	A–T–L; 229	A=229; los demás valores se conservan

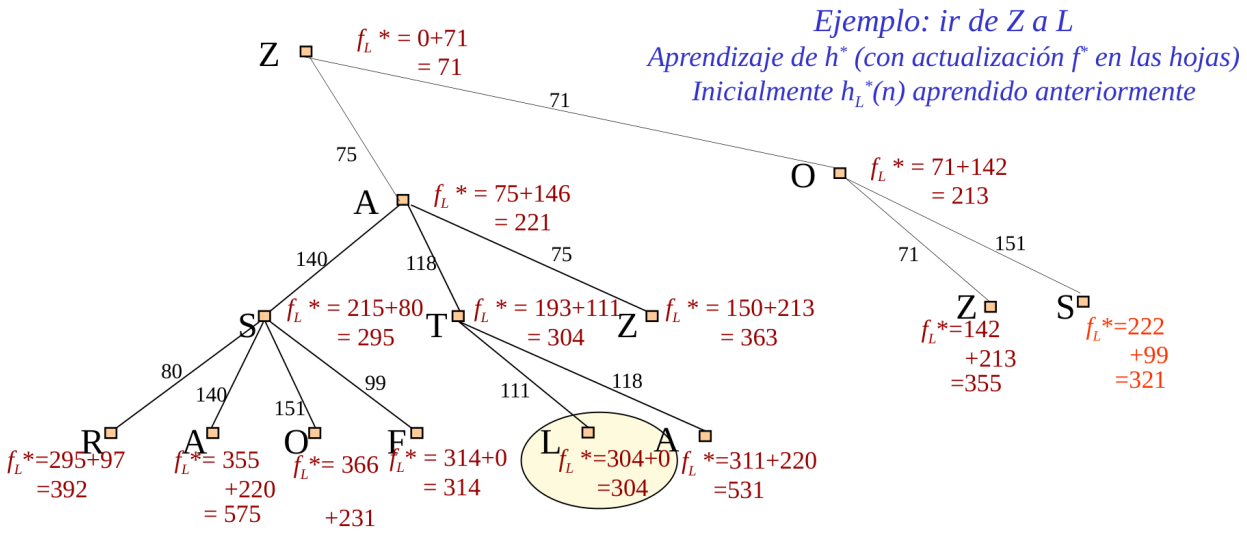


Figura 4. Segunda búsqueda Z→L: las estimaciones aprendidas se usan desde otro origen. Recorte docente.

En la tercera búsqueda, A evalúa Z con 75+213=288, T con 118+111=229 y S con 140+99=239. Aprende 229, selecciona T y después L. La ruta no mejora respecto a la primera: ya era óptima. Lo que se reduce es el trabajo para encontrarla, de siete expansiones no meta a dos. Los estados no aprendidos siguen con cero; no se confunde esta tabla parcial con un mapa exacto de distancias.

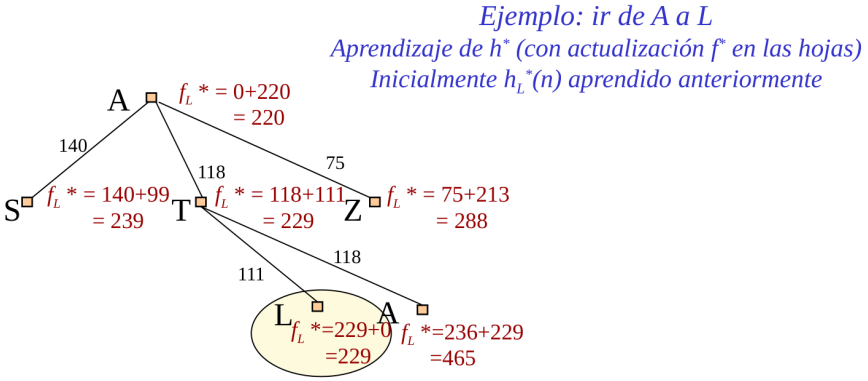


Figura 5. Tercera búsqueda A→L: la información acumulada concentra la expansión en el camino óptimo. Recorte docente.

3. Laberintos: heurística y aprendizaje

El ejercicio 3 utiliza una cuadrícula 3×3, movimientos ortogonales de coste 1 y barreras. Numeraremos las posiciones por (fila,columna), empezando en 1 arriba a la izquierda. La salida está en (1,3). El inicio de 3b es (1,2), mientras que el de 3c es (2,2): son dos búsquedas distintas.

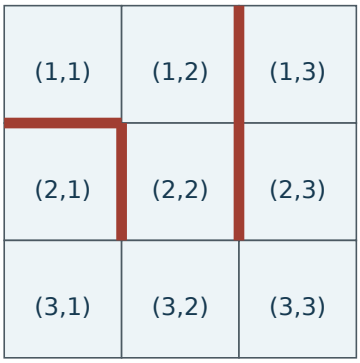


Figura 6. Reconstrucción del laberinto original con coordenadas. Las líneas gruesas son barreras; la salida es (1,3).

3.1. Manhattan ignora las barreras, pero sigue siendo válida

$$h^*((r,c)) = |r-1| + |c-3|.$$

Esta distancia es exacta en la cuadrícula relajada sin barreras. Las barreras pueden obligar a dar un rodeo, nunca crear un camino más corto. Así, h* es optimista. En cada movimiento ortogonal la distancia cambia exactamente en +1 o −1; su descenso máximo es 1, igual al coste. Por tanto es consistente. Desde (1,2) vale 1, aunque la barrera impide ir directamente a la salida y el camino real mínimo cuesta 5.

Información del agente. El enunciado dice que el agente desconoce las barreras, pero pide un árbol A* construido con los sucesores reales. La resolución presupone que las barreras se revelan al consultar/expandir un estado, o usa el laberinto como modelo para el ejercicio. Un agente físico que sólo percibe barreras donde está necesita especificar exploración y movimiento; el árbol offline del solucionario no describe por sí solo esa ejecución física.

3.2. A* sin eliminar estados repetidos

Se conserva cada llegada por separado. Esto explica que volver a (1,2) pueda ser seleccionado varias veces. El siguiente orden reproduce la solución; los empates restantes se resuelven como en su figura. En la tabla, cada hijo aparece con (g,h*,f*).

Paso	Posición	g, h*, f*	Sucesores
1	(1,2)	0, 1, 1	(1,1): (1,2,3); (2,2): (1,2,3)
2	(2,2)	1, 2, 3	(1,2): (2,1,3); (3,2): (2,3,5)
3	(1,2)	2, 1, 3	(1,1): (3,2,5); (2,2): (3,2,5)
4	(1,1)	1, 2, 3	(1,2): (2,1,3)

Paso	Posición	g, h*, f*	Sucesores
5	(1,2)	2, 1, 3	(1,1): (3,2,5); (2,2): (3,2,5)
6	(3,2)	2, 3, 5	(2,2): (3,2,5); (3,1): (3,4,7); (3,3): (3,2,5)
7	(3,3)	3, 2, 5	(2,3): (4,1,5); (3,2): (4,3,7)
8	(2,3)	4, 1, 5	(1,3): (5,0,5); (3,3): (5,2,7)
9	(1,3)	5, 0, 5	Meta: finalizar

El camino solución es (1,2)→(2,2)→(3,2)→(3,3)→(2,3)→(1,3), de coste 5. La secuencia de expansiones no es el camino: incluye regresos que sólo son alternativas del árbol. El nodo meta es el noveno seleccionado; se han expandido ocho nodos no meta.

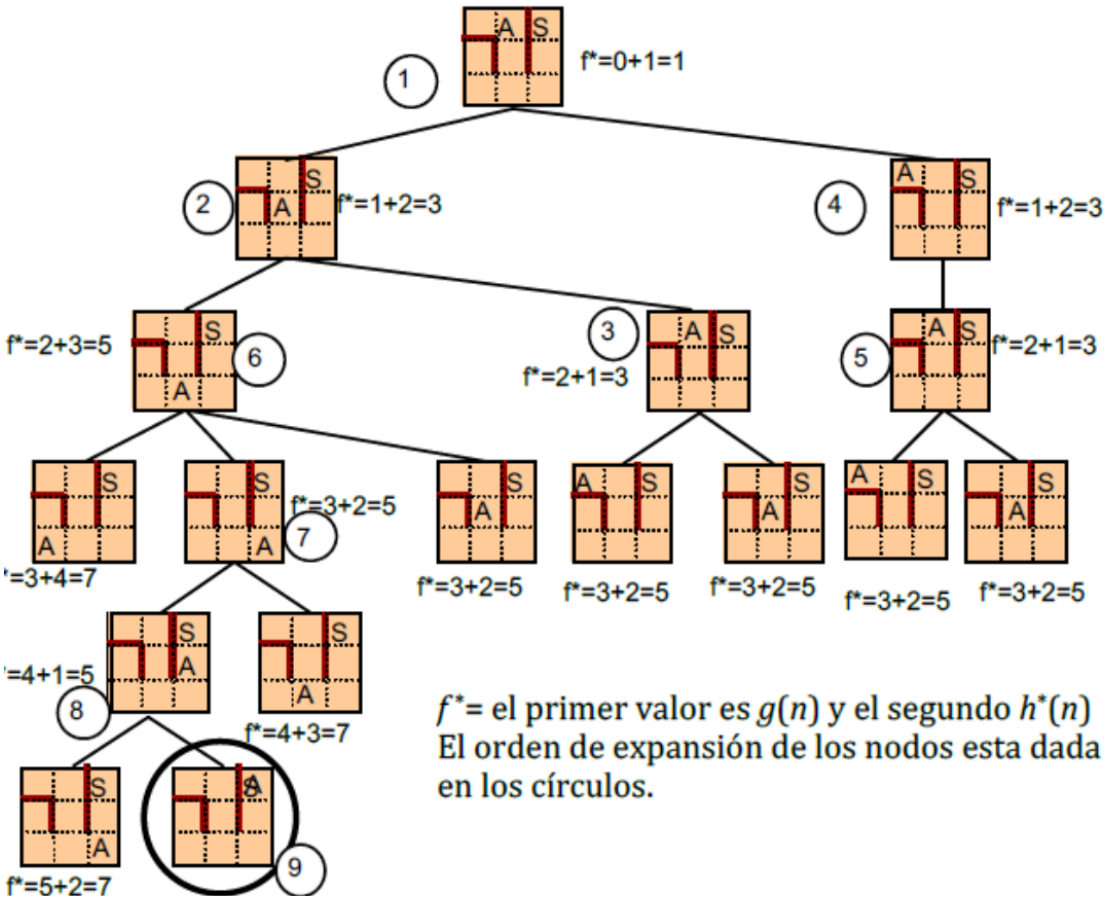


Figura 7. Árbol original de A* con Manhattan y repeticiones. Los círculos indican el orden que se desglosa en la tabla. Recorte del solucionario.

3.3. Aprender desde cero sin actualizar las hojas antiguas

Ahora el inicio es (2,2) y todas las estimaciones son cero. La tabla compartida se actualiza al expandir, pero los nodos que ya estaban abiertos conservan su prioridad antigua. La ruta óptima cuesta 4. Las dos primeras expansiones aprenden h*((2,2))=1 y h*((3,2))=1. Al expandir (1,1), su único sucesor (1,2) ya vale 1, de modo que aprende 1+1=2.

Paso	Estado	f* al extraer	h* en tabla	Hijos y f*
1	(2,2)	0+0=0	0 → 1	(1,2): 1+0=1; (3,2): 1+0=1
2	(3,2)	1+0=1	0 → 1	(2,2): 2+1=3; (3,1): 2+0=2; (3,3): 2+0=2
3	(1,2)	1+0=1	0 → 1	(1,1): 2+0=2; (2,2): 2+1=3
4	(3,3)	2+0=2	0 → 1	(2,3): 3+0=3; (3,2): 3+1=4
5	(3,1)	2+0=2	0 → 1	(2,1): 3+0=3; (3,2): 3+1=4
6	(1,1)	2+0=2	0 → 2	(1,2): 3+1=4
7	(2,3)	3+0=3	0 → 1	(1,3): 4+0=4; (3,3): 4+1=5
8	(2,2)	2+1=3	1 → 2	(1,2): 3+1=4; (3,2): 3+1=4
9	(2,2)	2+1=3	2 → 2	(1,2): 3+1=4; (3,2): 3+1=4
10	(2,1)	3+0=3	0 → 2	(3,1): 4+1=5
11	(1,3)	4+0=4	0 (meta)	Finalizar

En el paso 8 se vuelve a expandir el centro, ahora con ambos sucesores de valor 1: aprende 2. La otra copia del centro, nodo 9, conserva f*=2+1=3 y todavía se expande. La tabla ya contiene 2, y el cálculo devuelve 2 otra vez: el paso 9 no mejora la heurística. En el paso 10, (2,1) aprende 2 porque su único sucesor, (3,1), vale 1.

Posición	Evolución de h*	Paso de actualización
(1,1)	0 → 2	6
(1,2)	0 → 1	3
(1,3)	0	Sin cambio
(2,1)	0 → 2	10
(2,2)	0 → 1 → 2	1, 8
(2,3)	0 → 1	7
(3,1)	0 → 1	5
(3,2)	0 → 1	2
(3,3)	0 → 1	4

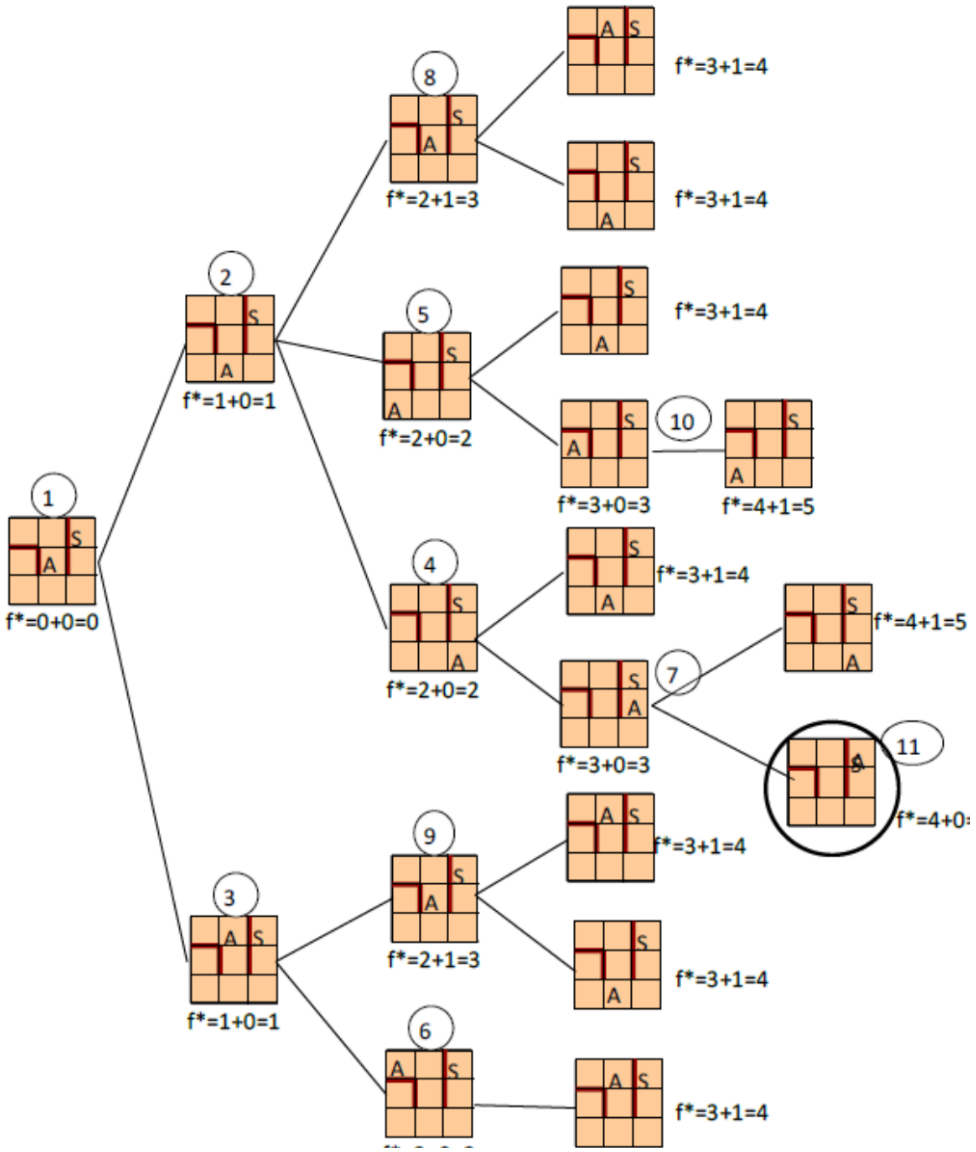


Figura 8. Árbol de aprendizaje original, sin actualización de f* en las hojas antiguas. Recorte del solucionario; la evolución de la tabla se reconstruye arriba.

Si se activa la extensión de las diapositivas, tras el paso 8 la prioridad de la copia 9 pasa a 2+2=4. A continuación se expande el nodo 10 con f*=3 y se selecciona la meta con f*=4 al preferir la menor distancia restante. No se expande la copia 9 antes de terminar. El aprendizaje del estado y la actualización de prioridades son operaciones relacionadas, pero distintas.

4. Diseño mediante problemas relajados

Una heurística puede proponerse por intuición o por analogía con otro dominio, pero necesita una justificación de sus propiedades. Un método sistemático consiste en resolver una versión más permisiva del problema y usar su coste óptimo como estimación para el original.

4.1. Qué se relaja y cómo se demuestra el optimismo

En la definición de las diapositivas, P y P′ mantienen los estados, el estado inicial y las metas. P′ permite todas las acciones de P y algunas adicionales, y conserva el coste de las acciones compartidas. Por tanto, todos los caminos solución originales siguen siendo legales en P′. Al minimizar sobre un conjunto mayor de caminos sólo puede disminuir el coste: $h_{P'}(s) \leq h_P(s)$.

$h^*_P(s) := h_{P'}(s)$.

La relajación es útil si obtener ese mínimo resulta mucho más barato que resolver P. Si también exigiera una búsqueda enorme, podría ser una buena cota y una mala elección computacional. Además, la distancia óptima relajada es consistente sobre las aristas originales: en P′ se puede usar primero s→t y después su mejor continuación, de modo que $h_{P'}(s) \leq c(s,t) + h_{P'}(t)$.

En una cuadrícula sin barreras con movimientos ortogonales unitarios, el coste exacto entre (u,v) y (x,y) es $|u-x|+|v-y|$. Si se añaden movimientos diagonales también de coste 1, pasa a ser $\max(|u-x|,|v-y|)$: se recorren en diagonal tantas diferencias de coordenadas como sea posible y se completa el resto en línea recta. Por ejemplo, diferencias 4 y 2 requieren 6 pasos ortogonales o 4 pasos con diagonales.

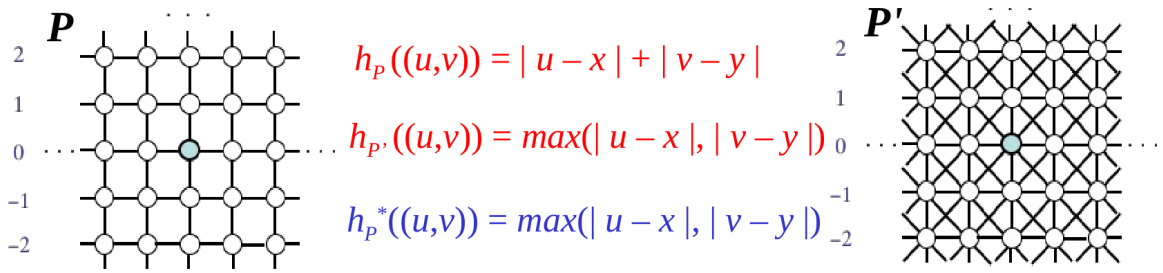


Figura 9. Cuadrícula original y relajación con diagonales unitarias. Recorte docente. El máximo de las diferencias es una cota inferior para la suma.

> Añadir restricciones suele aumentar el coste óptimo; quitarlas suele disminuirlo. No se debe usar una distancia que supone movimientos más caros como cota inferior sin demostrarlo. Manhattan deja de ser optimista si el problema real permite diagonales de coste 1: una diagonal cuesta 1 y Manhattan estimaría 2.

4.2. El 8-puzzle y sus tres relajaciones

Un estado guarda la posición de las ocho fichas y del hueco. Una acción mueve una ficha adyacente al hueco a esa posición, con coste 1. Según dónde esté el hueco existen 2, 3 o 4 movimientos posibles. Las dos precondiciones son C_1: el destino Y está vacío, y C_2: el origen X y el destino Y son adyacentes ortogonales.



Figura 10. Tableros reconstruidos a partir de los objetos visibles de las diapositivas 12–13. El hueco no se cuenta como ficha.

Problema relajado	Condición conservada	Coste exacto usado como h*
P′a	Ninguna de C_1 y C_2	h*a: número de fichas descolocadas. Cada una se coloca directamente en un movimiento.
P′b	Sólo C_1: destino vacío	h*b: mínimo número de intercambios entre una ficha cualquiera y el hueco; se permiten saltos no adyacentes.
P′c	Sólo C_2: adyacencia	h*c: suma de las distancias Manhattan de las fichas a sus destinos, al ignorar ocupación e interferencias.

Para entender P′a y P′c, se ignoran las interferencias entre fichas: cada una puede evaluarse independientemente aunque otra ocupe su destino. Eso corresponde a una relajación/abstracción de las restricciones de ocupación; no debe interpretarse como un movimiento legal del puzzle físico. La definición literal de «mismo conjunto de estados» exige formalizar ese modelo auxiliar si se admiten ocupaciones superpuestas. La cota también se demuestra directamente contando movimientos necesarios de cada ficha.

En el tablero visible, las fichas descolocadas son 1, 2, 6, 7 y 8. Sus distancias Manhattan son respectivamente 1, 1, 1, 3 y 1: h*a=5 y h*c=7. Una acción real sólo mueve una ficha una casilla; puede reducir Manhattan como máximo en 1, y puede corregir como máximo una ficha descolocada. Ambas funciones son consistentes y optimistas.

4.3. Cómo calcular los saltos de h*b

La expresión «suma de saltos» significa resolver el puzzle relajado que permite intercambiar el hueco con cualquier ficha. Si el hueco está en una casilla cuyo destino es una ficha, se trae esa ficha. Si el hueco ya está en su destino y quedan fichas descolocadas, se intercambia con una de ellas para abrir el ciclo pendiente, y se continúa. Se cuentan los intercambios, no las distancias geométricas.

En el ejemplo docente, el hueco inicial está abajo en el centro. Se intercambia sucesivamente con 6, 7, 2, 1 y 8: cinco saltos. Los estados son los siguientes; algunos saltos no serían legales en el puzzle real.

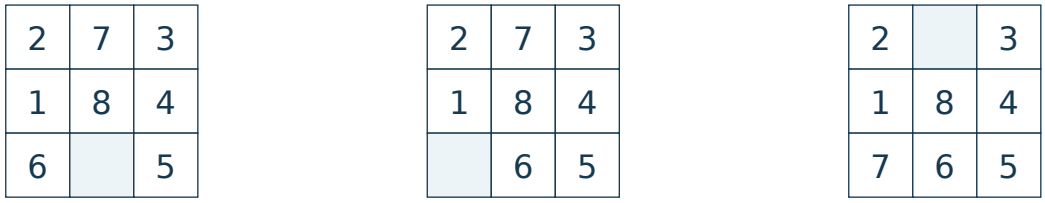


Figura 11. Primeros estados de la relajación de saltos; reconstrucción nueva.



Figura 12. Final del proceso: cinco intercambios llevan a la meta.

En general, h*b no es siempre igual a las fichas descolocadas. Si el hueco está correcto y sólo dos fichas están intercambiadas, hacen falta tres saltos: sacar una al hueco, colocar la otra y devolver la primera. Contar dos descolocadas infravalora esos tres saltos. Esta observación explica por qué no debe sustituirse h*b por h*a aunque en la diapositiva ambas valgan 5.

5. Una h* más informada: significado y límites

Esta sección desarrolla la explicación adicional solicitada sobre «h-estrella-mas-informada». Para dos heurísticas optimistas, h^*_1 es más informada que h^*_2 cuando ofrece una cota al menos tan alta en **todos** los estados. El coste real sigue siendo el techo permitido.

$$\forall s:\ h^*_2(s) \le h^*_1(s) \le h(s).$$

La definición docente usa ≥, no >. Permite que las heurísticas sean iguales en muchos estados e incluso en todos; «más informada» se entiende aquí como «al menos tan informada». Superar a otra en un único estado no demuestra dominancia. Por ejemplo, si en X los valores son 6 y 3, pero en Y son 2 y 5, ninguna domina a la otra.

5.1. Por qué Manhattan domina a las piezas descolocadas

Cada ficha bien colocada aporta cero a ambas sumas. Cada ficha descolocada aporta uno a h*a y al menos uno a h*c, porque necesita separarse de su posición actual al menos una casilla para llegar a la meta. Sumando ficha por ficha se obtiene $h^*c(s) \ge h^*a(s)$ para cualquier tablero. La prueba cubre todos los estados, no sólo el ejemplo 5 frente a 7.

Una cota mayor hace menos atractivas las ramas cuyo coste real restante es grande. Por ejemplo, con g(X)=4 y g(Y)=3, una heurística débil que dé h*(X)=1 y h*(Y)=1 prefiere Y: f*(Y)=4 frente a 5. Si otra heurística optimista da 1 y 6, respectivamente, pasa a preferir X: 5 frente a 9. No ha encarecido los caminos reales; ha reducido la infravaloración de Y.

5.2. Demostración de la idea de dominancia

Sea C* el coste de una solución óptima. En una meta óptima, ambas heurísticas valen cero, así que ambas evaluaciones coinciden con C*. En un mismo nodo del árbol, g es el mismo y $h^*_1 \geq h^*_2$ implica $f^*_1 \geq f^*_2$. Por ello, si f^*_1 está por debajo de C*, también lo está f^*_2.

$$f^*_1(n) < C^* \Rightarrow f^*_2(n) < C^*.$$

Para contemplar heurísticas no monótonas en un árbol, la condición debe afectar a todos los prefijos: un nodo bajo el umbral puede permanecer oculto detrás de un antecesor por encima del umbral. Si todos los prefijos tienen $f^*_1 < C^*$, también tienen $f^*_2 < C^*$. La región de caminos obligatorios bajo el umbral con la heurística más informada queda incluida en la de la menos informada. Con consistencia, f* no disminuye por el camino y este razonamiento se simplifica.

Las diapositivas 17–22 ilustran cuatro situaciones: ambas curvas permanecen bajo el umbral; sólo la curva más informada lo supera en un prefijo; ambas lo superan; o la supuesta curva más informada queda por debajo de la otra en algún punto. La última situación está tachada porque contradice la hipótesis de dominancia, no porque el camino no exista.

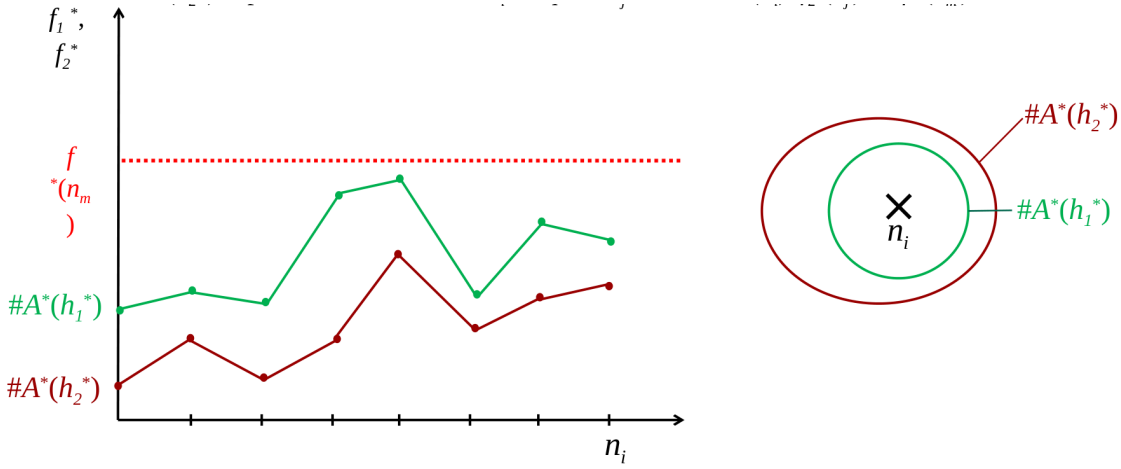


Figura 13. Ambas evaluaciones quedan bajo C*: el camino es candidato con las dos heurísticas. Recorte de la secuencia gráfica docente.

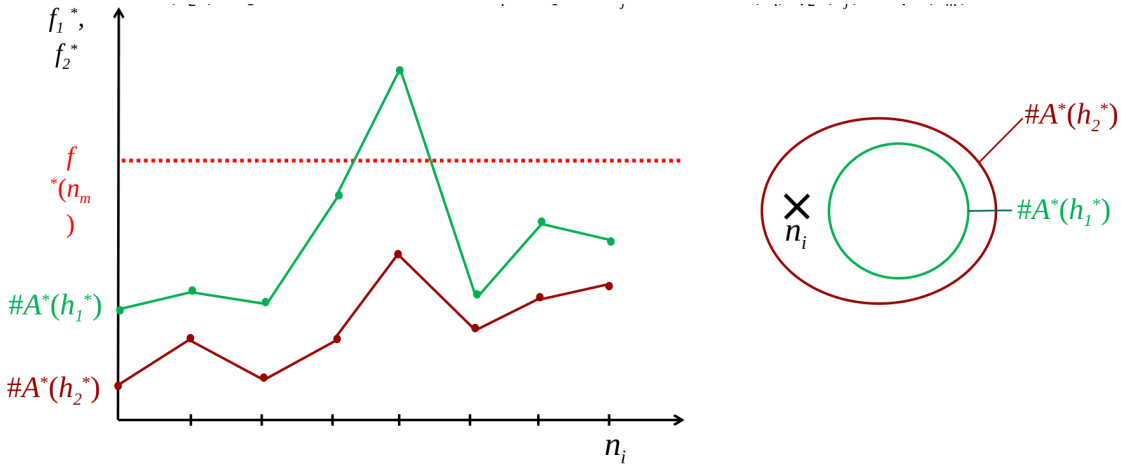


Figura 14. Sólo la curva más informada supera C*: puede evitarse ese camino con ella. Recorte de la secuencia gráfica docente.

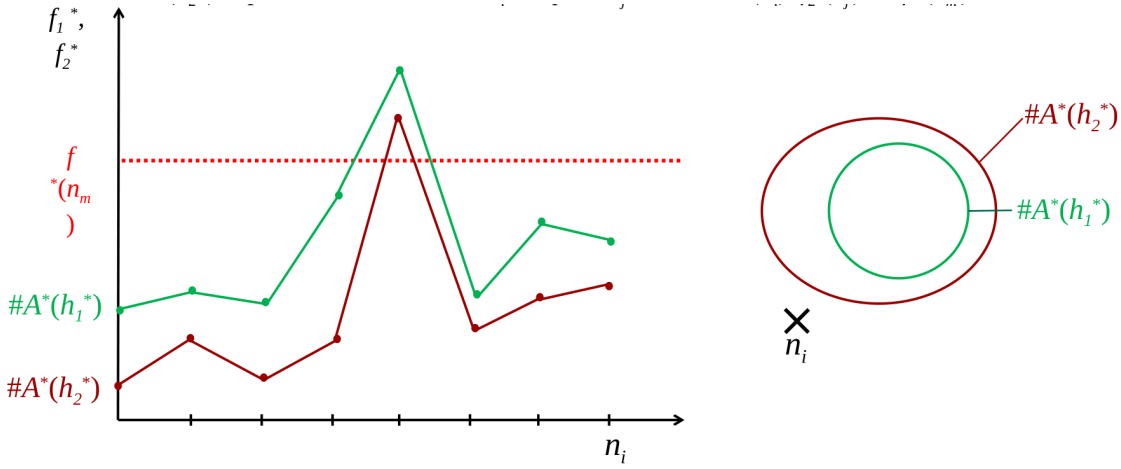


Figura 15. Ambas curvas superan C*: ninguna necesita seguir ese prefijo antes de la solución óptima. Recorte de la secuencia gráfica docente.

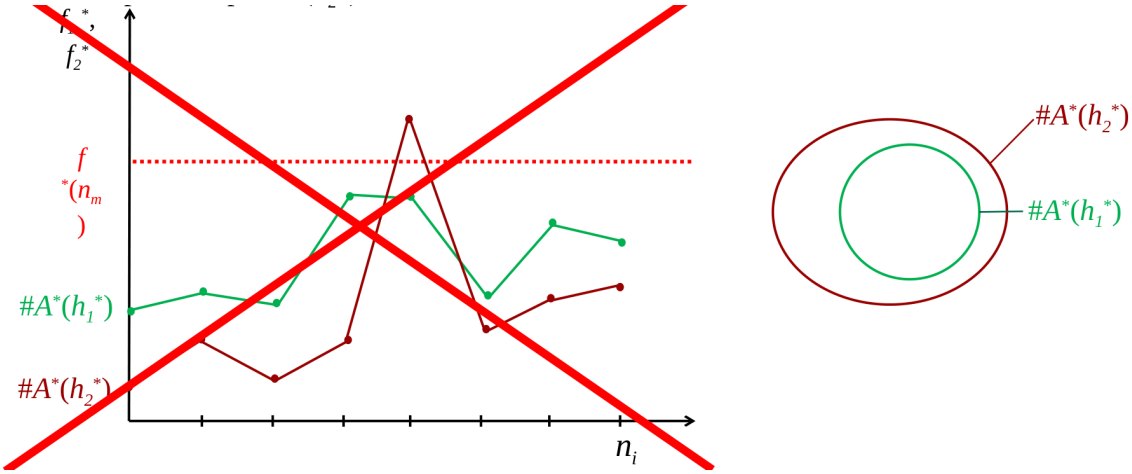


Figura 16. Caso incompatible con dominancia: las curvas se cruzan en el sentido prohibido. Recorte de la secuencia gráfica docente.

5.3. La frontera de empate no se puede ignorar

Precisión respecto al teorema 3. Las diapositivas usan «≤C*» como si todos esos nodos se expandieran obligatoriamente. Lo obligatorio corresponde a «<C*» en las condiciones anteriores. Con f*=C* hay empates: algunos nodos se expanden y otros quedan abiertos al seleccionar una meta. La afirmación sobre el número total exacto requiere precisar desempates, variante de A* y posibles reaperturas. [R2]

Un contraejemplo sencillo muestra el problema de universalizar el conteo. S tiene dos hijos, una meta G y un nodo X, ambos a coste 1; X lleva a otra meta a coste 1. Con h*=0, G y X empatan en f*=1. Una ejecución puede seleccionar G inmediatamente, mientras que otra puede seleccionar X antes de G. Las heurísticas son idénticas y, por la definición ≥, cada una domina a la otra; sin embargo, las expansiones difieren si difiere el desempate.

La interpretación útil para estudiar es: una heurística dominante reduce la región que A* debe explorar por tener f* estrictamente menor que C*. Suele reducir expansiones, pero no cambia el coste óptimo encontrado. Tampoco garantiza menor tiempo si calcularla es mucho más caro. La comparación debe fijar costes de evaluación, criterio de desempate, filtrado de estados y criterio de conteo.

5.4. Combinar información con el máximo

$$h^*(s) = \max(h^*_1(s), h^*_2(s), \ldots, h^*_m(s)).$$

Si todas son optimistas, cada valor es $\leq h(s)$, así que el máximo también lo es. Por construcción domina a cada componente. Si una componente domina globalmente a otra, ésta se puede omitir del máximo. Si sólo la supera en ciertos estados, no puede eliminarse en todo el dominio. El máximo conviene cuando las componentes se evalúan con un coste razonable.

Si todas son consistentes, el máximo también: sea j una componente que alcanza el máximo en s. Entonces $h^*(s)=h^*_j(s)\leq c(s,t)+h^*_j(t)\leq c(s,t)+\max_i h^*_i(t)$. Si alguna es inconsistente, el máximo sigue siendo optimista, pero no se garantiza consistencia. Una heurística inconsistente no se descarta automáticamente: puede aportar información útil si A* maneja las reaperturas necesarias.

No se suman heurísticas arbitrariamente. Dos estimadores pueden contar los mismos movimientos. Si ambos estiman 4 y el coste real es 5, el máximo 4 es admisible, pero la suma 8 no. La suma requiere una justificación específica de que los costes contados no se duplican.

5.5. Comprobaciones conceptuales del ejercicio 1

Ítem	Respuesta	Justificación
1.1a	Verdadero	Con costes no negativos, $0\leq h(n)$.
1.1b	Verdadero en la convención docente	Las heurísticas son no negativas y «más informada» usa ≥, incluyendo igualdad. Si se admitieran valores negativos, habría que matizarlo.
1.1c	Verdadero con el mismo desempate y filtrado	Con $h^*=0$, ordenar por f^* equivale a ordenar por g: coste uniforme.
1.1d	Falso	Una heurística informativa puede evitar expansiones que hace coste uniforme.
1.2a	Verdadero	La componente dominada no aumenta el máximo. Se conserva la gestión correcta de repetidos si la dominante es inconsistente.
1.2b	Falso	Una desigualdad local no establece dominancia global.
1.2c	Verdadero	El máximo de estimadores optimistas fáciles de evaluar conserva el optimismo.
1.2d	Falso	Inconsistente no significa inadmisible ni inútil.

6. Aplicación: comparar A* en el 8-puzzle

El ejercicio 2 cambia el tablero inicial respecto a las diapositivas. Ahora es 123 / 64_ / 875 y la meta sigue siendo 123 / 8_4 / 765. En la escritura por filas, «_» es el hueco. Se eliminan ciclos generales y, si hay empate, se escoge antes la alternativa fuera del camino óptimo. Las dos heurísticas se calculan sin contar el hueco.



Figura 17. Tableros del ejercicio reconstruidos. El coste óptimo es 5 movimientos.

6.1. Piezas descolocadas: entender el empate decisivo

En el inicio están fuera de lugar 4, 6, 7 y 8: h*a=4 y f*=4. Mover la ficha 4 al hueco produce 123 / 6_4 / 875 con g=1, h*a=3 y f*=4. Sus sucesores no repetidos incluyen 123 / _64 / 875 y 123 / 674 / 8_5, ambos con g=2, h*a=3 y f*=5. El primero pertenece a la solución óptima; la condición del peor caso obliga a expandir primero el segundo.

De la rama incorrecta se generan dos sucesores con f*=7 y f*=6, de modo que A* vuelve a la rama correcta de f*=5. Ésta mueve el hueco hacia abajo, a la derecha y arriba para alcanzar la meta. Se observa la utilidad de separar coste acumulado y estimación: el valor f* permanece en 5 mientras g sube y h* baja.

Selección	Estado	g	h*a	f*
1	123 / 64_ / 875	0	4	4
2	123 / 6_4 / 875	1	3	4
3	123 / 674 / 8_5	2	3	5
4	123 / _64 / 875	2	3	5
5	123 / 864 / _75	3	2	5
6	123 / 864 / 7_5	4	1	5
7	123 / 8_4 / 765	5	0	5

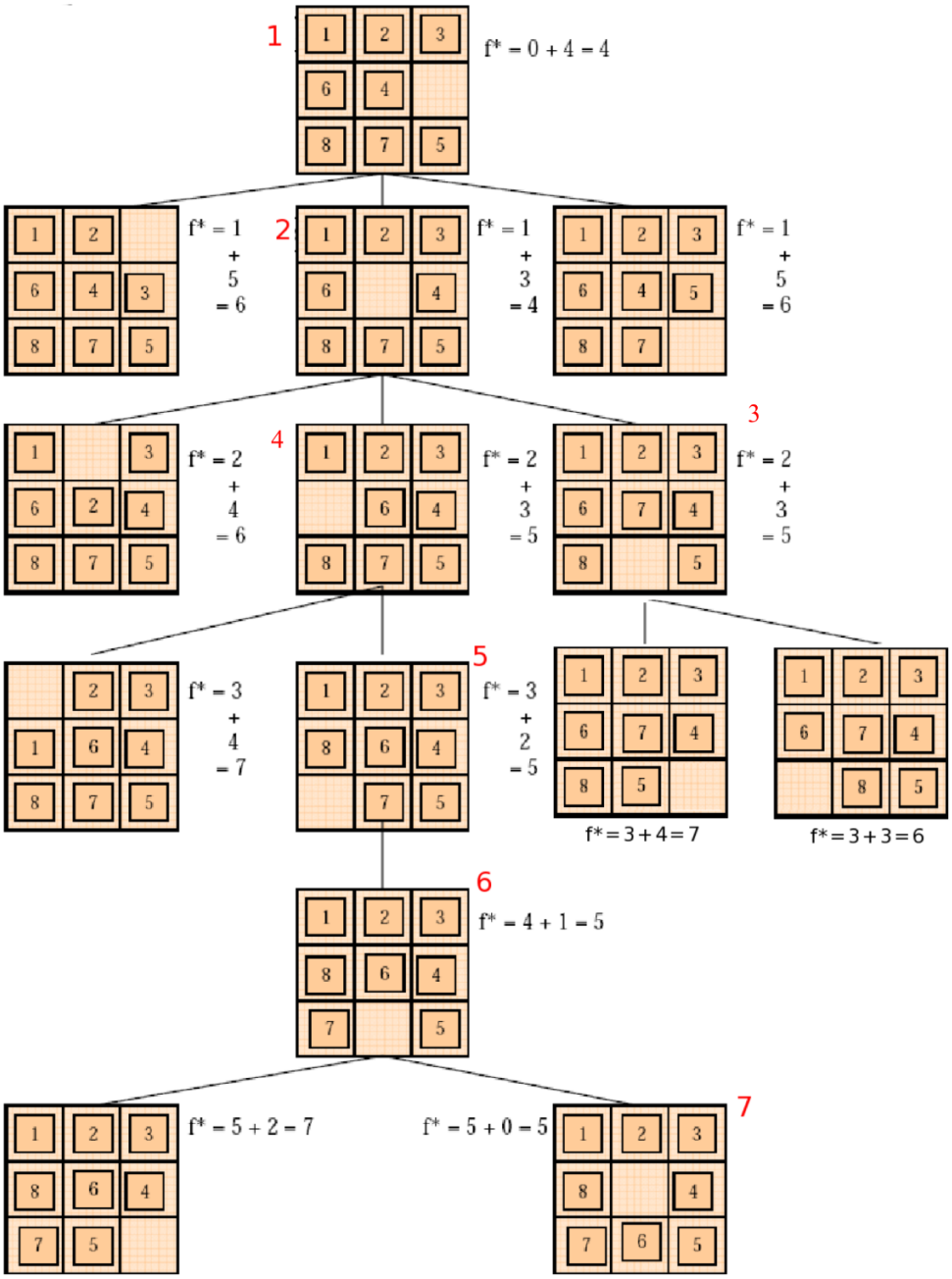


Figura 18. Árbol original con piezas descolocadas. Se conservan todos los hijos no repetidos y sus valores; la meta es la selección 7. Recorte del solucionario.

6.2. Manhattan: por qué desaparece esa expansión

En el inicio, la ficha 6 necesita dos pasos y las fichas 4, 7 y 8 uno cada una: h*c=5. Tras el primer movimiento, el estado del centro vale 4. Al comparar las dos ramas antes empatadas, la correcta tiene h*c=3 y f*=5; la incorrecta tiene h*c=5 y f*=7. La diferencia elimina el empate y esa rama queda abierta sin expandirse antes de la meta.

Selección	Estado	g	h*c	f*
1	123 / 64_ / 875	0	5	5
2	123 / 6_4 / 875	1	4	5

Selección	Estado	g	h*c	f*
3	123 / _64 / 875	2	3	5
4	123 / 864 / _75	3	2	5
5	123 / 864 / 7_5	4	1	5
6	123 / 8_4 / 765	5	0	5

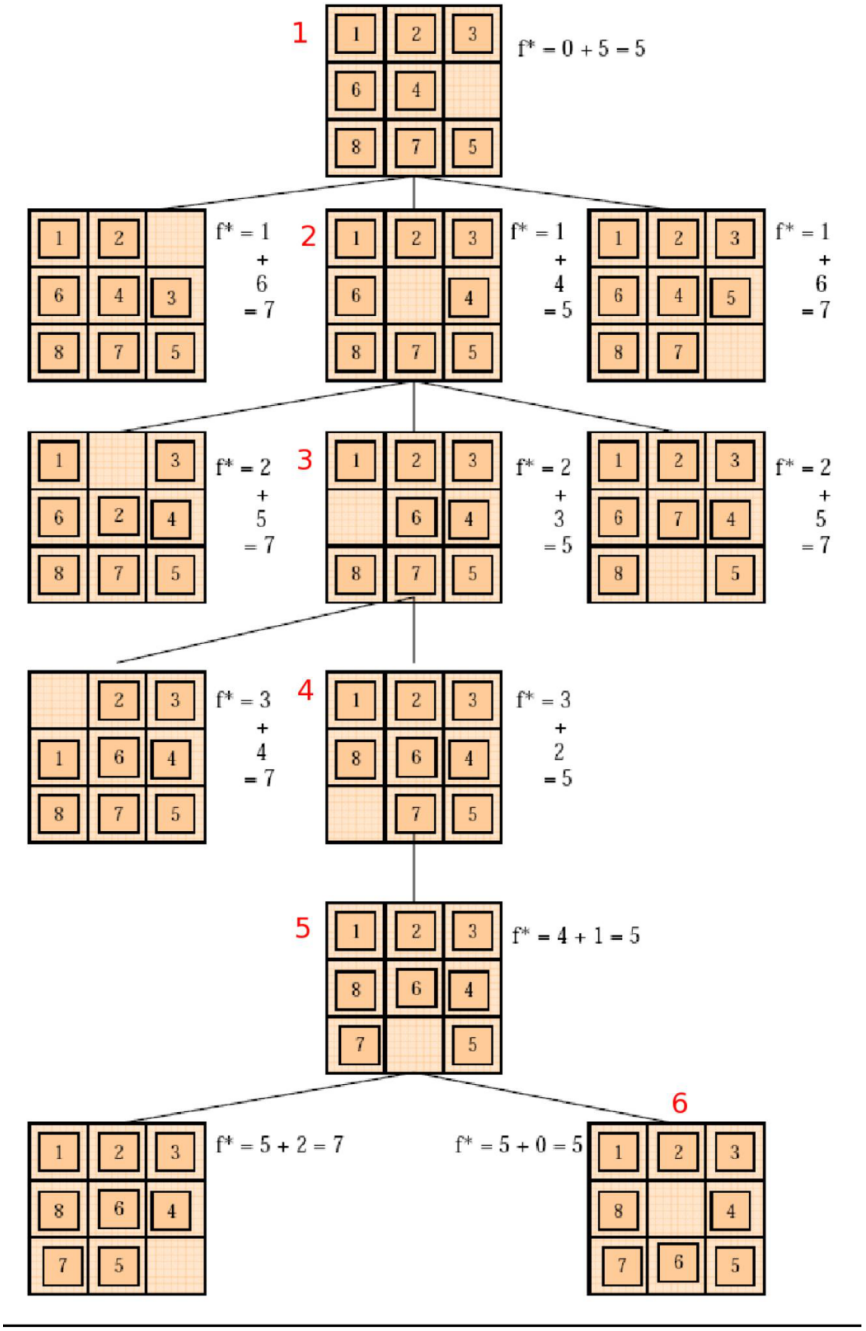


Figura 19. Árbol original con Manhattan, con todos los sucesores no repetidos. La meta es la selección 6. Recorte del solucionario.

La ruta óptima, expresada como movimientos del hueco, es izquierda, izquierda, abajo, derecha, arriba. Cuesta 5 con ambas heurísticas. La primera genera 14 nodos contando la raíz y selecciona 7; la segunda genera 12 contando la raíz y selecciona 6. Si se

cuentan sólo expansiones que producen hijos, son 6 y 5. Estos valores se han contrastado con las trazas del solucionario y con una búsqueda independiente.

Para resolver una variante, no memorices el árbol. Calcula la heurística para la meta concreta, conserva g, genera únicamente movimientos legales, aplica el filtrado solicitado y anota qué nodos empatan en el mínimo global de Abierta. «Peor caso» no permite elegir un nodo con f* mayor cuando hay otro con f* menor.

7. Modelar bien importa tanto como la heurística

7.1. Cuadrado de suma 15: representación eficiente

El ejercicio 4 lo llama «cuadrados latinos», pero sus reglas describen un **cuadrado mágico normal 3×3**: usar una vez cada entero de 1 a 9 y hacer que las tres filas, tres columnas y dos diagonales sumen 15. Un cuadrado latino tiene otra definición. Se conserva el nombre del enunciado al referirse al ejercicio, pero se resuelven sus reglas explícitas.

Sea N={1,...,9} y T=N∪{_}. Un estado es un vector de longitud 9 sobre T en el que no se repite ningún número; los huecos sí pueden repetirse. El inicio general tiene nueve huecos. Una meta debe estar completa y satisfacer las ocho sumas. Elegir siempre el menor número aún no usado evita representar diferentes órdenes de inserción que conducen al mismo tablero final.

$$n'(s) = \min(N \setminus \text{números usados en } s).$$

Cada sucesor coloca n′ en uno de los huecos. Se genera uno por hueco y el coste es 1. Un tablero completo no tiene sucesores: si incumple las sumas, es un callejón sin salida. No se calcula el mínimo de números no usados cuando ya se usaron todos. El orden fijo de números no pierde soluciones, porque cualquier tablero final se puede construir colocando 1, luego 2, etc., en las posiciones que le correspondan.

1	9	2
7	5	6
8	4	3

Ejemplo completo: h*=6

2		4
	5	3
6	1	

Inicio parcial: h*=7

Figura 20. Los dos tableros del enunciado. En el primero sólo suman 15 la última fila y la diagonal 2–5–8.

7.2. A* con el número de líneas incumplidas

La heurística propuesta cuenta filas, columnas y diagonales cuya suma actual no es 15; los huecos no aportan valor. En el inicio parcial sólo la diagonal 4+5+6 cumple la suma, así que h*=7. Faltan 7, 8 y 9. Primero se genera una alternativa por cada colocación posible del 7.

Nodo	Tablero por filas	g	h*	f*
A	2_4 / _53 / 61_	0	7	7
B	274 / _53 / 61_	1	7	8
C	2_4 / 753 / 61_	1	5	6
D	2_4 / _53 / 617	1	7	8
E	2_4 / 753 / 618	2	2	4
F	284 / 753 / 61_	2	5	7
G	294 / 753 / 618	3	0	3

C coloca el 7 en el hueco central izquierdo, corrige dos líneas y obtiene f*=1+5=6. Es preferible a B y D, que valen 8. Desde C se coloca el 8 abajo a la derecha (E) o arriba en el centro (F). E deja sólo dos líneas incumplidas y vale 2+2=4. Finalmente se coloca el 9 en el único hueco restante y aparece G con coste 3.

Momento	Abierta ordenada por f*
Inicial	A(7)
Tras expandir A	C(6), B(8), D(8)
Tras expandir C	E(4), F(7), B(8), D(8)
Tras expandir E	G(3), F(7), B(8), D(8)
Seleccionar G	Meta con g=3; finalizar

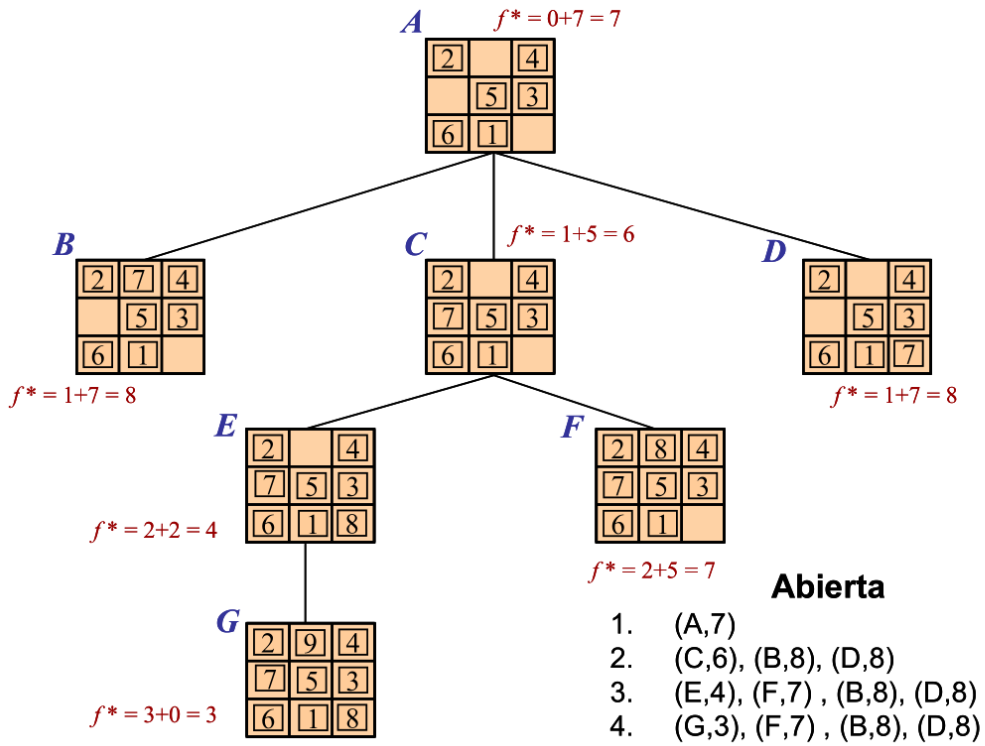


Figura 21. Árbol y lista Abierta originales del cuadrado de suma 15. Recorte del solucionario.

7.3. Una heurística no admisible puede acertar en un problema

En E queda un solo movimiento hasta G, pero h*(E)=2>h(E)=1: la heurística no es optimista. También viola directamente la consistencia en E→G: 2≤1+0 es falso. Colocar una ficha puede corregir varias líneas simultáneamente; contar líneas no equivale a contar movimientos necesarios.

Sin embargo, toda solución desde el inicio parcial necesita exactamente tres inserciones. Desde el tablero vacío necesitaría nueve. Como no se borran fichas y cada acción llena un hueco, todas las soluciones tienen igual coste. Si A* encuentra una, será de coste mínimo en este problema, incluso con esta heurística. Esto no convierte la heurística en optimista ni demuestra una garantía general para otros dominios.

Una cota alternativa sería el número de huecos: coincide con el coste restante de cualquier solución alcanzable. También en estados sin solución sigue siendo una cota inferior de +∞. Este ejemplo muestra cómo una representación y un coste particular pueden simplificar una prueba de optimalidad.

8. Rutas con batería: ampliar el estado

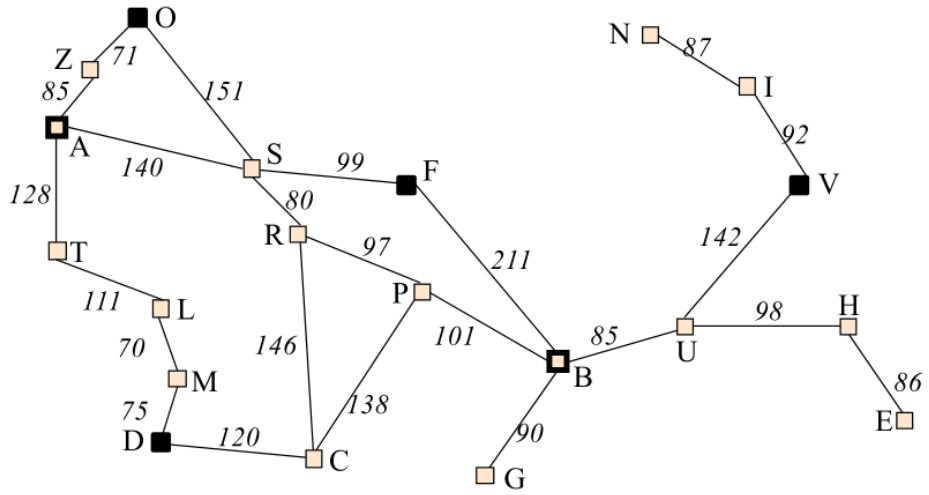
En el ejercicio 5, el coche parte de A con autonomía para 320 km y busca B por la menor distancia total. Recarga en D, F, O y V. La carga no añade distancia al objetivo del ejercicio. El estado debe guardar la ciudad y la autonomía restante: dos llegadas a la misma ciudad con baterías distintas no permiten las mismas acciones.

8.1. Definición formal y relajación

$S = P \times \{0,1,\ldots,320\}$; $s_0=(A,320)$; $\text{meta}(p,b) \Leftrightarrow p=B$.

P es el conjunto de ciudades. El solucionario usa P×ℕ; restringirlo a 0…320 hace explícito el límite físico y no cambia las transiciones alcanzables. Se permite ir de (p,b) a una ciudad adyacente q sólo si d(p,q)≤b. Si q tiene estación, el sucesor es (q,320); si no, es (q,b−d(p,q)). El coste de esa transición es d(p,q), independientemente de la carga final.

Se supone recarga completa automática y sin coste de distancia, como en el solucionario. Si se optimizara tiempo, precio o desgaste, habría que incluir los costes y decisiones de recarga. El estado ciudad por sí solo sería insuficiente incluso en el problema actual: la llegada a P con 3 km no permite recorrer sus aristas salientes.



	h^*
A	366
B	0
C	160
D	242
E	161
F	178
G	77
H	151
I	226
L	244
M	241
N	234
O	380
P	98
R	193
S	253
T	329
U	80
V	199
Z	374

Figura 22. Mapa del coche eléctrico y tabla de distancias aéreas a B. Recorte del enunciado. Las estaciones son D, F, O y V.

> **No mezclar mapas.** Aquí A–Z=85 y A–T=128. En las diapositivas de aprendizaje y horizonte son 75 y 118. El resto de cálculos debe seguir el mapa concreto de cada ejemplo, aunque las ciudades tengan el mismo nombre.

La heurística es h*((p,b))=distancia aérea(p,B), que no depende de b. Ignorar la batería relaja el problema: permite rutas que el coche no podría realizar. Toda ruta factible sigue siendo una ruta por carreteras y su longitud es al menos la distancia aérea. Así se justifica el optimismo. Si la tabla representa distancias geométricas y las carreteras no son más cortas que la distancia aérea entre extremos, la desigualdad triangular también da consistencia.

8.2. Expansión completa sin filtrar ciclos

Desde A, los valores son S:140+253=393, T:128+329=457 y Z:85+374=459. Se expande S. Sus sucesores incluyen R con batería 100 y f*=413, F con recarga completa y f*=417, A con batería 40 y f*=646, y O con recarga completa y f*=671. Se conserva el retorno a A porque no se filtran ciclos.

Paso	Estado	g, h*, f*	Sucesores legales
1	(A,320)	0, 366, 366	(Z,235): 85+374=459; (T,192): 128+329=457; (S,180): 140+253=393
2	(S,180)	140, 253, 393	(A,40): 280+366=646; (O,320): 291+380=671; (R,100): 220+193=413; (F,320): 239+178=417
3	(R,100)	220, 193, 413	(S,20): 300+253=553; (P,3): 317+98=415
4	(P,3)	317, 98, 415	Sin sucesores factibles
5	(F,320)	239, 178, 417	(S,221): 338+253=591; (B,109): 450+0=450
6	(B,109)	450, 0, 450	Meta: terminar

R puede llegar a P consumiendo 97 de sus 100 km, por lo que P queda con 3. Su f*=317+98=415 parece favorable, pero no puede realizar ninguna transición. El siguiente mínimo es F. Allí se dispone de 320 km; recorrer F–B consume 211 y deja 109. La ruta óptima factible es A–S–F–B, de distancia 140+99+211=450 km.

Momento	Abierta (estado: f*)
Inicio	(A,320):366
Tras A	(S,180):393; (T,192):457; (Z,235):459
Tras S	(R,100):413; (F,320):417; (T,192):457; (Z,235):459; (A,40):646; (O,320):671
Tras R	(P,3):415; (F,320):417; (T,192):457; (Z,235):459; (S,20):553; (A,40):646; (O,320):671
Tras P	(F,320):417; (T,192):457; (Z,235):459; (S,20):553; (A,40):646; (O,320):671
Tras F	(B,109):450; (T,192):457; (Z,235):459; (S,20):553; (S,221):591; (A,40):646; (O,320):671

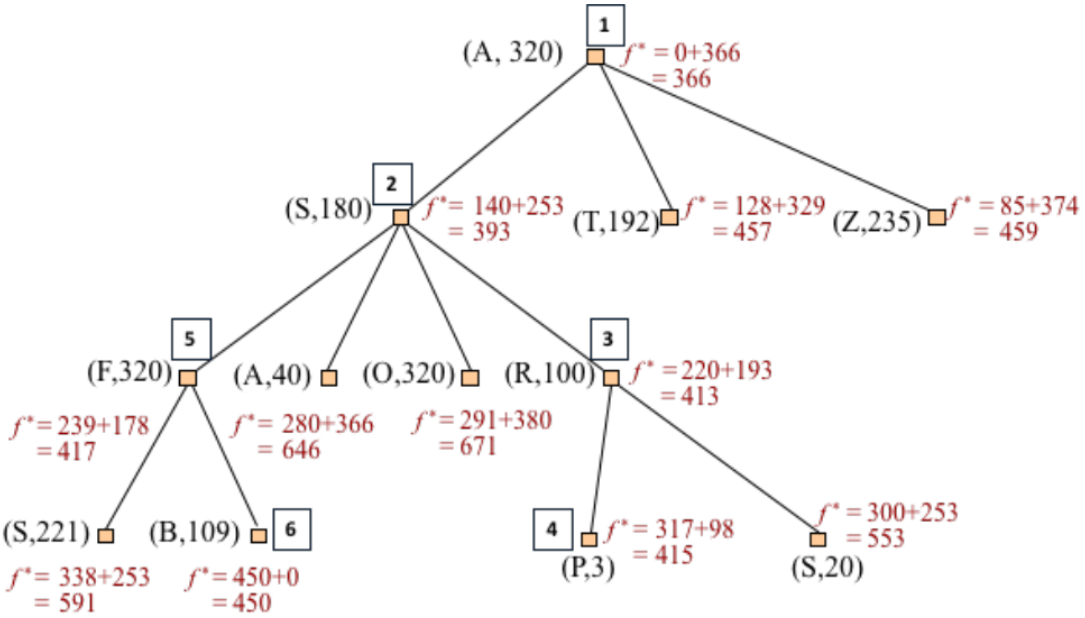


Figura 23. Árbol original del coche eléctrico, con cada estado (ciudad,batería) y sus costes. Recorte del solucionario.

El camino A–S–R–P–B, de 418 km en el mapa habitual, falla por autonomía: llega a P con 3 km y necesita otros 101. A* no puede tratar una estimación prometedora como autorización para ejecutar una acción ilegal. Si se quisiera filtrar estados, habría que comparar pares (ciudad,batería); una dominancia por coste y batería sería una optimización adicional, ausente del ejercicio.

9. Complejidad y alternativas con menos memoria

9.1. Precisión heurística y coste de computación

La heurística afecta a cuántos nodos se exploran y al trabajo invertido en evaluarlos. Con $h^*=0$, A* se convierte en coste uniforme. Con $h^*=h$ se dispone del coste restante exacto, pero calcularlo suele equivaler a resolver el problema. No es válido declarar eficiente un algoritmo que esconde todo el trabajo dentro del cálculo de h^*.

La diapositiva 24 presenta la heurística perfecta como caso lineal y relaciona complejidad con error absoluto $|h-h^*|$: una cota $O(c)$ para comportamiento lineal y $O(\log h)$ para comportamiento polinomial. El símbolo c no queda definido con precisión allí; la lectura pedagógica es un error acotado por una constante. Estas expresiones no constituyen teoremas universales para cualquier espacio de estados: faltan hipótesis sobre ramificación, costes, estructura del espacio y desempates.

Contraejemplo a «perfecta implica siempre lineal». En un árbol binario completo de profundidad d con todas las hojas meta y coste 1 por arista, la heurística exacta es d−profundidad. Todos los nodos tienen $f^*=d$. Con empate por anchura, A* puede expandir 2^d-1 nodos antes de seleccionar una meta; con preferencia por profundidad puede seguir un solo camino. El error heurístico es cero en ambos casos.

En situaciones reales el error puede crecer con la distancia restante. Reducirlo suele mejorar mucho respecto a búsqueda no informada, sin eliminar el peor caso exponencial. Si una heurística nueva divide las expansiones entre dos pero cuesta cien veces más evaluarla, el tiempo puede empeorar. A* también mantiene una frontera grande: la memoria suele ser el límite antes que el tiempo.

9.2. La tabla experimental del tema

Las diapositivas comparan el promedio sobre 100 instancias del 8-puzzle por profundidad de solución d. La tabla se encontraba en un documento incrustado en la diapositiva 25; se ha recuperado y reproducido. Los guiones significan que no se ofrece dato, no cero expansiones.

d	Profundidad iterativa no informada	A*(h*a)	A*(h*c)
2	10	6	6
4	112	13	12
6	680	20	18
8	6.384	39	25

d	Profundidad iterativa no informada	A*(h*a)	A*(h*c)
10	47.127	93	39
12	3.644.035	227	73
14	—	539	113
16	—	1.301	211
18	—	3.056	363
20	—	7.276	676
22	—	18.094	1.219
24	—	39.135	1.641

A profundidad 24, la media con piezas descolocadas es unas 23,85 veces la de Manhattan (39.135/1.641). El resultado ilustra una mejora práctica de exploración, no demuestra una nueva clase de complejidad ni mide por sí solo el tiempo de ejecución. Las fuentes no proporcionan las 100 instancias ni el protocolo completo, por lo que se verifican los datos recuperados y su interpretación, no se reproduce el experimento.

9.3. IDA*, SMA* y RBFS

La diapositiva 26 denomina «heurísticas débiles» a mejoras que intentan conservar la optimalidad atacando la memoria, a costa de regenerar algunos nodos. La explicación siguiente amplía los nombres y la idea general que allí se enumeran; no presupone que el profesor exija desarrollar sus implementaciones.

Método	Cómo limita memoria	Qué sacrifica o condiciona
IDA* · Korf (1985)	Hace profundidad primero con un límite en f*, inicialmente h*(inicio). En cada iteración usa como próximo límite el menor f* que excedió el anterior.	Repite exploración. El umbral es coste estimado, no profundidad. Conserva optimalidad bajo las hipótesis adecuadas de admisibilidad y búsqueda.
SMA* · Russell (1992)	Usa la memoria disponible. Cuando debe descartar hojas poco prometedoras, guarda información de sus valores en los padres para poder regenerarlas.	Puede repetir mucho trabajo. Su garantía de encontrar una solución óptima depende de disponer de memoria suficiente para representarla.
RBFS · Korf (1992)	Explora recursivamente la mejor alternativa con un límite derivado de la siguiente mejor; conserva valores resumidos de alternativas y retrocede si supera el límite.	Ahorra memoria respecto a guardar toda la frontera, pero puede regenerar subárboles al volver a ellos.

Con ramificación acotada, una búsqueda en profundidad puede almacenar esencialmente el camino actual y sus alternativas, en lugar de todos los nodos de una frontera exponencial. No debe confundirse «menor memoria» con «menos expansiones». Las referencias docentes para ampliar son Russell y Norvig, 3.ª edición, §3.5.3, y

Nilsson, §§9.2.4–9.2.5; aquí se citan como referencias que aparecen en la presentación, sin afirmar haber revisado esas páginas.

10. Restringir la búsqueda: islas y jerarquías

10.1. Subobjetivos o islas

Una heurística fuerte toma decisiones que acotan el espacio considerado. En la búsqueda por islas se fijan estados $i_1,\ldots,i_n$ y se resuelven por separado los tramos inicio→i_1→…→i_n→meta. Puede ser útil si ciertos puntos son buenos pasos intermedios, por ejemplo puertos de montaña. Sin una garantía de que una ruta óptima pase por esas islas en ese orden, la suma de tramos óptimos no tiene por qué ser globalmente óptima.

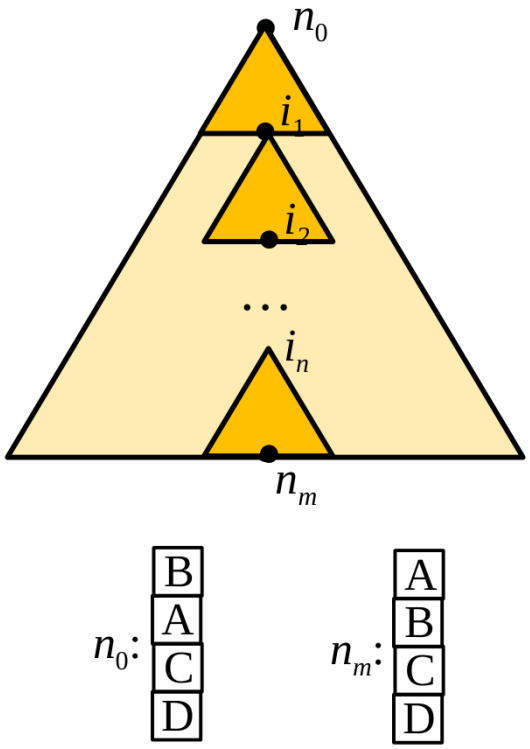


Figura 24. Islas que restringen la búsqueda y ejemplo del mundo de bloques. Dos recortes de los diagramas docentes.

El ejemplo docente parte de una torre B sobre A sobre C sobre D y quiere A sobre B sobre C sobre D. Con un operador que mueve un bloque libre a la mesa o sobre otro bloque libre, basta: B a la mesa, A a la mesa, B sobre C y A sobre B. Son cuatro movimientos. Si se impone la isla «todos los bloques en la mesa», primero hay que desmontar B, A y C (tres movimientos), y luego montar C sobre D, B sobre C y A sobre B (otros tres): seis en total. La isla obliga a deshacer la relación C sobre D, que ya era correcta.

10.2. Búsqueda jerárquica

En vez de fijar las islas manualmente, se define un espacio a un metanivel. Un estado puede ser un país y un macrooperador «pasar al país vecino». A* busca una secuencia de países; después se refina cada transición en rutas concretas por ciudades o carreteras. Es posible repetir el proceso con varios niveles de abstracción.

El plan abstracto organiza los subobjetivos, pero no garantiza por sí solo que las transiciones refinadas existan o que sus costes sean los estimados. Puede requerir retroceso o cambio de plan abstracto. La ventaja es reducir el detalle explorado de golpe; la limitación es que una elección abstracta puede descartar una solución mejor o incluso una solución factible.

11. Búsqueda en línea y horizonte limitado

11.1. Planificar y actuar se alternan

En búsqueda offline se construye primero un plan en un modelo y después se ejecuta. En búsqueda online se percibe el estado actual, se busca lo necesario para decidir la siguiente acción, se ejecuta físicamente y se vuelve a percibir. Se aplica cuando el espacio es demasiado grande, el modelo es desconocido o los efectos previstos sufren contingencias, como un brazo que deja caer un bloque.

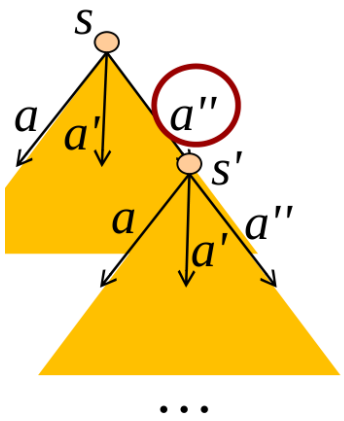


Figura 25. La búsqueda se reinicia desde el estado al que realmente llega el agente. Recorte del esquema docente.

> Índice competitivo = coste del recorrido real / coste del camino óptimo.

Cuanto menor sea el cociente, mejor; 1 significa igual coste que el óptimo, si éste es positivo. Desvíos y retrocesos físicos cuentan en el numerador. En general no se garantizan ni optimalidad ni completitud. Una acción irreversible hacia un callejón sin salida puede impedir llegar a la meta y hacer el índice infinito, aunque hubiera una ruta óptima finita. Si el coste óptimo es cero, el cociente exige un tratamiento específico.

Ítem del ejercicio 1.3	Respuesta y motivo
a	Verdadero: alterna búsqueda con acción/percepción.
b	Falso: no hay garantía general de completitud ni optimalidad.
c	Verdadero: un horizonte permite decidir sin buscar de una vez todo el camino.
d	Falso: interesa minimizar el índice competitivo.

11.2. Qué significa mirar k acciones hacia delante

Desde el estado actual s se construye un árbol limitado a k acciones. Para cada nodo de frontera n se calcula $f^*_s(n)=g_s(n)+h^*(n)$, donde g_s mide el coste desde el estado actual, no desde el inicio de todo el viaje. Se elige la mejor continuación estimada y se ejecuta sólo su primera acción. Después se descarta o reconstruye la búsqueda local desde el nuevo estado físico.

H_k = nodos alcanzados a profundidad k; $n^* \in \arg\min_{n\in H_k} [g_s(n)+h^*(n)]$.

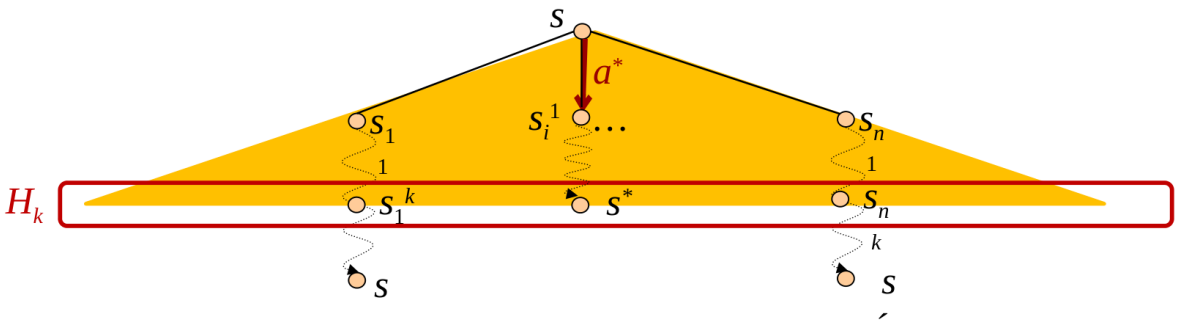


Figura 26. Frontera H_k y primera acción del camino elegido. Recorte docente. El agente no ejecuta de golpe todas las acciones simuladas.

Explorar más pasos sustituye una parte de la estimación por costes reales del modelo. Puede mejorar la decisión, pero no garantiza que el error disminuya en cada nodo más profundo. Con consistencia, las cotas f* no disminuyen por un camino; la optimalidad global del recorrido online sigue sin estar garantizada. Se está suponiendo que la mejor hoja local anticipa una buena decisión futura.

11.3. Algoritmo local y casos que hay que especificar

Se inicia $\alpha=+\infty$ y se recorre el árbol con profundidad limitada. Cada hoja válida del horizonte actualiza $\alpha=\min(\alpha,f^*)$. Se recuerda el primer candidato que alcanza el mejor valor; con empates se mantiene el orden de exploración establecido. Se ejecuta la primera acción hacia ese candidato y se repite hasta estar físicamente en una meta. El límite k cuenta acciones, no kilómetros. Al cambiar la raíz, g_s vuelve a cero.

> **Ambigüedad de la fuente.** La diapositiva 33 dice que, si se encuentra una meta, se elija esa meta. Sin embargo, en la 35 se descubre B con coste 310 y después se prefiere la hoja no meta P con f*=275. La ejecución numérica corresponde a comparar también las metas por su coste contra las hojas del horizonte, sin prioridad absoluta. Aquí se explica esa variante para reproducir el ejemplo. Con prioridad absoluta se escogería S→F y el recorrido posterior podría ser distinto.

Una versión operativa coherente con las figuras trata como candidatos las metas encontradas hasta profundidad k y las hojas no meta de profundidad k, comparándolas por f*. Un nodo muerto no meta sin sucesores no ofrece una continuación válida y se descarta. Si no queda ningún candidato, el método necesita declarar fallo local o una política de recuperación. Un horizonte mayor o un mecanismo de aprendizaje/retroceso puede ayudar, pero no forma parte automáticamente de esta regla.

11.4. Poda α: cuándo se puede cerrar una rama

Si h* es consistente, f*(descendiente)≥f*(n′) para todo descendiente de n′. Una vez conocido un candidato de valor α, una rama cuyo nodo interior satisface f*(n′)≥α no puede aportar un candidato estrictamente mejor. Se puede podar si sólo se desea conservar el primer mínimo. Si se quieren todas las alternativas empatadas, no se elimina la igualdad sin más.

h* consistente y f*(n′) ≥ α ⇒ no hay mejora estricta bajo n′.

Con una heurística inconsistente, un hijo puede tener menor f* que su padre. Por ejemplo, con g(n′)=2, h*(n′)=8, una arista de coste 1 y h*(hijo)=0, el padre vale 10 y el hijo 3. Podar el padre por superar α=5 perdería una mejora. La regla necesita monotonía, no sólo que parezca razonable.

11.5. Ejemplo completo: A → S → R → P → B

Se usa k=2, costes del mapa de las diapositivas y filtrado de retornos inmediatos dentro del árbol local. La búsqueda en profundidad sigue el orden visual de izquierda a derecha. La tabla siguiente recoge las hojas evaluadas, las podas interiores y el movimiento real elegido. Los valores f* se calculan desde la raíz local de cada fila.

Estado real	Evaluación local y evolución de α	Acción
A	Por S: F=239+178=417; O=291+380=671; R=220+193=413. α: ∞→417→413. T tiene f*=447 y Z=449: se podan antes de llegar al horizonte.	A→S
S	Por A: T=258+329=587; Z=215+374=589. Por F: B=310. O tiene f*=531≥310: poda. Por R: C=226+160=386; P=177+98=275. α: ∞→587→310→275.	S→R
R	Por C: D=266+242=508; P=284+98=382. Por P: B=198; C=235+160=395. S tiene f*=333≥198: poda. α: ∞→508→382→198.	R→P
P	B es meta a profundidad 1 con f*=101. C tiene 138+160=298 y R, 97+193=290. La meta ya domina estas ramas.	P→B

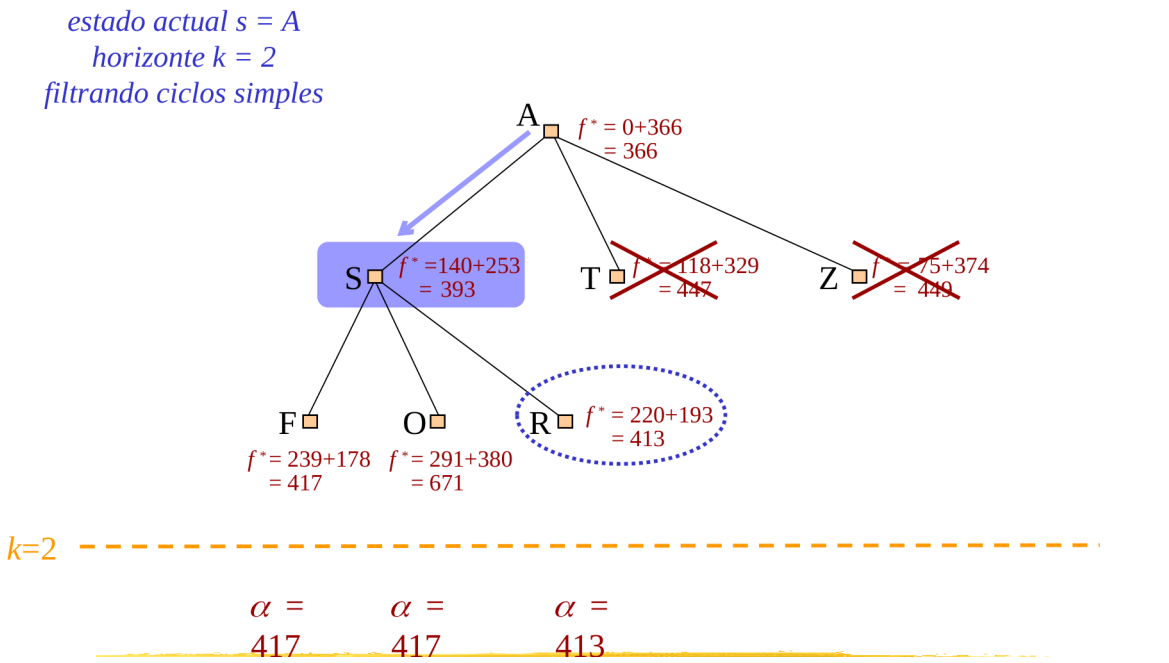


Figura 27. Desde A: la mejor hoja local es R; sólo se ejecuta A→S. Recorte del árbol y de sus podas.

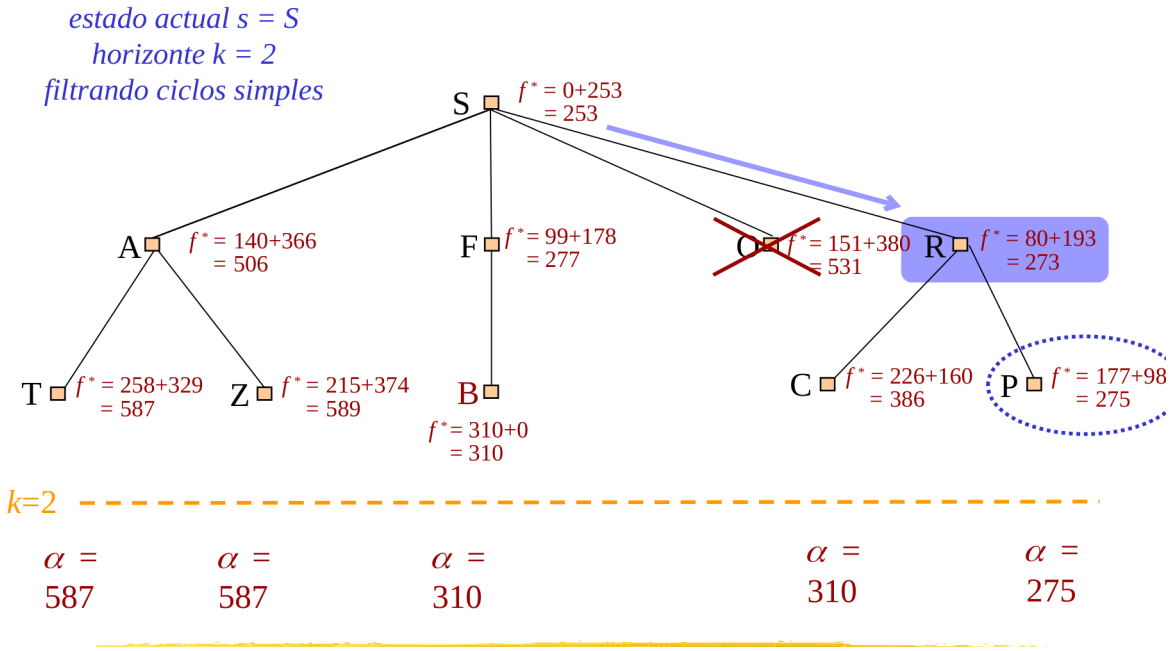


Figura 28. Desde S: P=275 mejora B=310; se ejecuta S→R en la variante mostrada. Recorte del árbol y de sus podas.

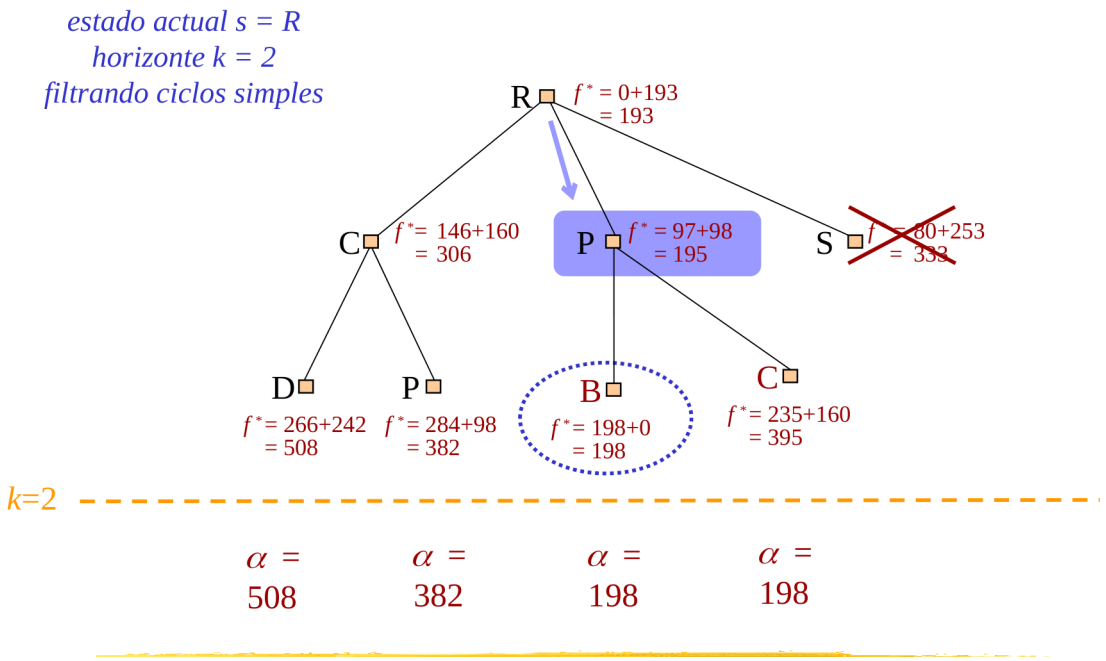


Figura 29. Desde R: la ruta local por P a B vale 198; se ejecuta R→P. Recorte del árbol y de sus podas.

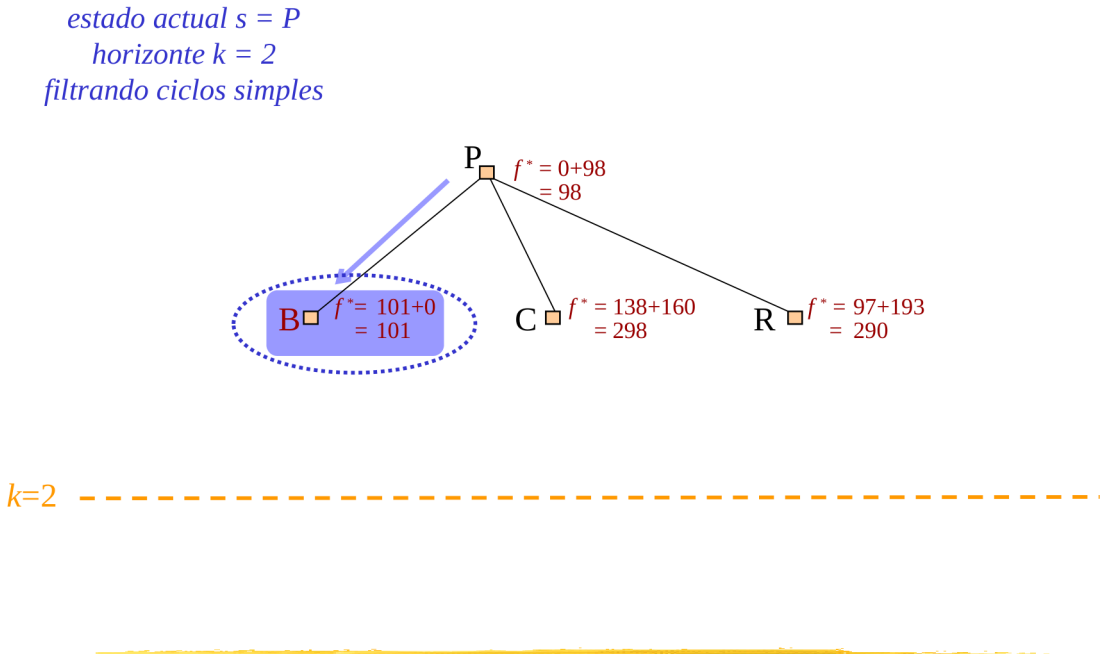


Figura 30. Desde P: B es una meta directa y se ejecuta P→B. Recorte del árbol y de sus podas.

El recorrido físico cuesta 140+80+97+101=418. En este mapa coincide con el óptimo a B y su índice competitivo es 1. Esa coincidencia es un resultado del ejemplo, no una garantía de búsqueda con horizonte. En cada nueva raíz, los costes se recalculan: P tiene g=177 desde S, pero g=97 desde R.

Es posible que un horizonte fijo oscile: en un pasillo con dos estados vecinos A y X y una salida más cara desde A, con h*=0 y k=1 puede elegirse A→X por ser el paso más barato y luego X→A por ser el único regreso. Si se olvida la historia al replanificar, el ciclo puede continuar. El filtrado de ciclos dentro de una búsqueda local no impide necesariamente ciclos entre decisiones físicas sucesivas.

12. Criterios para resolver y comprobar un problema nuevo

Primero identifica qué información condiciona las acciones: posición, tablero, batería u otros recursos. Define el estado completo, la meta y el coste de una acción. Después separa g de h*: g se acumula por el camino y h* estima lo que queda hasta la meta concreta. Si cambias la meta, no reutilices sin justificación una tabla aprendida para otra.

Para demostrar optimismo, compara con una relajación o con un coste mínimo inevitable. Para demostrar consistencia, verifica $h^*(s) - h^*(t) \leq c(s,t)$ para una transición arbitraria; un solo contraejemplo basta para refutarla. Para demostrar dominancia hace falta una desigualdad en todos los estados; un ejemplo aislado sólo sirve para refutar o ilustrar.

En cada traza de A*, anota Abierta antes de seleccionar, el criterio de empate, g de cada hijo, la heurística usada en ese instante y el filtrado aplicado. En aprendizaje, añade una tabla por estado y declara si se actualizan las prioridades de copias antiguas. En horizonte, anota raíz actual, k, frontera, α, ramas podadas y primera acción física, sin confundirla con la hoja que motivó la decisión.

La comprobación final tiene tres partes distintas: el camino debe ser legal; la suma de costes debe coincidir con g de la meta; y la afirmación de optimalidad debe apoyarse en propiedades y condiciones válidas. Una solución óptima en un ejemplo no prueba admisibilidad de la heurística, como muestra el cuadrado mágico. Una heurística más precisa no garantiza menor tiempo, como muestra el coste de evaluarla.

Fuentes y precisiones de lectura

Material principal. «00 Diapositivas-Tema4_con_animacion.ppsx», 37 diapositivas, y «02 Ejercicios-Tema4-Solución.pdf», 11 páginas. La autoría y el contexto institucional de los gráficos recortados corresponden al material docente URJC. Los diagramas nuevos y las explicaciones ampliadas se identifican en sus pies o contexto.

[R1] Poole y Mackworth, Artificial Intelligence: Foundations of Computational Agents, 3.ª ed., §3.6. Consulta para condiciones de A*, admisibilidad y papel de los empates: texto académico abierto.

[R2] Corrêa, Pereira y Ritt (2018), Analyzing Tie-Breaking Strategies for the A* Algorithm, IJCAI, pp. 4715–4721. Consulta para distinguir calidad heurística y estrategias de desempate: artículo y resumen oficial.

[R3] UC Berkeley, CS188, §1.4, Informed Search. Contraste de admisibilidad, dominancia y máximo de heurísticas: apuntes oficiales. El desarrollo numérico y las pruebas del manual se construyen sobre los materiales adjuntos.

Alcance comprobado. Se inspeccionaron las 37 diapositivas y las 11 páginas, incluidos diagramas, objetos incrustados y estructura de animaciones. Se reconstruyen los pasos relevantes; no se afirma haber reproducido la presentación en PowerPoint ni haber visto los vídeos. El informe de auditoría separado identifica discrepancias, verificaciones y límites.